

LAPORAN TUGAS BESAR

Sistem Terdistribusi Dasar

Milestone 1

Platform Koordinasi Kebencanaan BNPB

Kelompok **AkuAnakTehat**

Dzaky Aurelia Fawwaz 13523065

Muhammad Alfansya 13523005

IF4031 Arsitektur Aplikasi Terdistribusi

8 Oktober 2026

Kontribusi Anggota

Pembagian berikut menjelaskan kontribusi setiap anggota berdasarkan komponen sistem dan tanggung jawab yang dikerjakan.

Aspek	Dzaky Aurelia Fawwaz (13523065)	Muhammad Alfansya (13523005)
Perancangan	Perancangan arsitektur sistem dan mekanisme penyelesaian P1 hingga P5.	Membantu verifikasi dan validasi kesesuaian rancangan terhadap spesifikasi.
Implementasi	BMKG Mock dan PVMBG Mock; Auth Service; polling, pemetaan, korelasi tsunami, transaksi dan watermark pada Aggregator; outbox relay dan producer Kafka; Dashboard Updater, Notifier, dan Pemda Portal; infrastruktur Docker Compose, bootstrap secret, dan CI; integrasi serta pengujian lintas komponen.	Query dan API internal Aggregator; Client API beserta rate limit, batas konkurensi, pagination, dan penanganan galat; CLI Tim Lapangan; serta script load test k6.
Penulisan laporan	Keseluruhan	Keseluruhan

Daftar Isi

Kontribusi Anggota	i
1 Deskripsi Umum dan Lingkup	1
1.1 Prinsip rancangan	1
2 Arsitektur Sistem	2
2.1 Alur kerja sistem	3
2.2 Komponen dan kepemilikan data	3
2.3 Protokol dan batas kepercayaan	4
3 Teknologi, Asumsi, dan Batasan	4
3.1 Teknologi terpilih	5
3.2 Asumsi operasional dan data	5
3.3 Batas layanan dan ketahanan	6
3.4 Batas ukuran dan pemulihan backlog	6
4 Petunjuk Menjalankan Sistem	7
5 P1. Arsitektur dan Interoperabilitas Data	7
5.1 Masalah dan mekanisme	7
5.2 Pemetaan kanonik	8
5.3 Deteksi perubahan dan transaksi	10
5.4 Alternatif dan trade-off	10
5.5 Bukti penyelesaian	11
6 P2. Konkurensi, Availability, dan Degradasi	11
6.1 Model konkurensi dan isolasi	11
6.2 Konfigurasi dasar	13
6.3 Kebaruan data dan kontrak galat	13
6.4 Konfigurasi dan hasil pengukuran	14
6.5 Outage berkepanjangan	16
6.6 Validitas hasil dan alternatif	16
7 P3. Autentikasi dan Domain Kepercayaan	17
7.1 Pemisahan kredensial	17
7.2 JWT, scope, dan proyeksi	17
7.3 Rotasi refresh dan batas atomisitas	18
7.4 Alasan pemilihan dan risiko yang ditangani	18

7.5	Bukti penyelesaian	20
8	P4. Batas Service dan Flexible Storage	20
8.1	Alasan menentukan batas service	20
8.2	Kepemilikan Canonical Store	21
8.3	Skema dan evolusi atribut	21
8.4	Perbandingan alternatif penyimpanan	22
8.5	Bukti independensi dan batas	22
9	P5. Distribusi Event dengan Consumer Independen	22
9.1	Broker dan pola yang dipilih	22
9.2	Outbox dan semantik pengiriman	23
9.3	Efek consumer, idempotensi, dan DLQ	24
9.4	Alternatif dan trade-off	26
9.5	Bukti penyelesaian	26
10	Ringkasan Verifikasi dan Batas Hasil	26
10.1	Skenario dan hasil akhir	27
10.2	Ketahanan ukuran event dan pemulihan	28
10.3	Verifikasi instrumentasi HTTP	28
10.4	Bootstrap pada volume kosong	28
10.5	Cakupan bukti dan pemeriksaan akhir	29
	Surat Pernyataan Penggunaan AI	30
A	Matriks Kriteria Spesifikasi	31
B	Indeks Bukti dan Jejak Eksekusi	33
B.1	Bukti pengujian akhir	33
B.2	Contoh trace event nyata	34
B.3	Asal potongan kode dan gambar	34
	Daftar Pustaka	35

1 Deskripsi Umum dan Lingkup

BNPB membutuhkan representasi yang konsisten atas informasi gempa, potensi tsunami, dan aktivitas gunung api dari dua instansi yang otonom. BMKG dan PVMBG memiliki skema, kredensial, serta karakteristik latensi berbeda. Sistem ini mengambil data melalui polling, memetakannya menjadi **HazardEvent**, menyajikan data sesuai hak akses client, dan menyalurkan perubahan melalui broker kepada consumer independen.

Kedua instansi berupa mock yang dibangun kelompok; data serta referensi gunung api bersifat sintetis. Tiga identitas downstream adalah Media, Tim Lapangan, dan BNPB Pusat. Seluruh akses downstream bersifat baca saja. Pelaporan dari lapangan, integrasi sistem instansi nyata, dan pengoperasian banyak instance tidak termasuk implementasi ini.

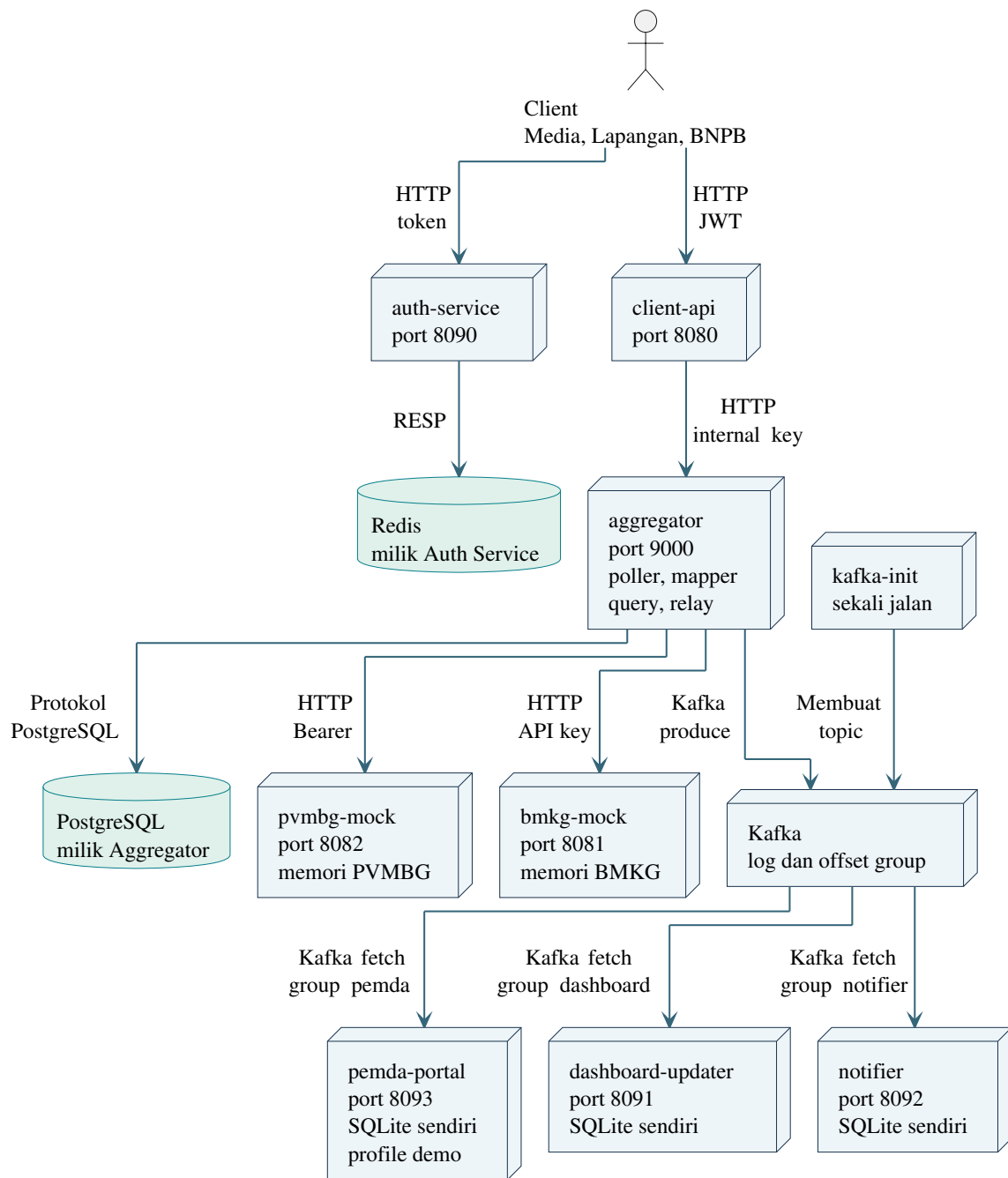
1.1 Prinsip rancangan

Sistem memisahkan proses ingest, pembacaan data, dan distribusi event. Proses ingest menangani ketidakpastian sumber, sedangkan pembacaan menggunakan data yang sudah tersimpan. Perubahan data selanjutnya diteruskan melalui outbox dan Kafka. Pemisahan ini membuat satu request client tidak harus menunggu HTTP ke instansi sumber. Hasil pembacaan karena itu dapat tertinggal dari kondisi terbaru di sumber. Waktu data dan status sumber disertakan agar pengguna dapat menilai kebaruannya.

Problem	Mekanisme inti yang diimplementasikan
P1	Tolerant reader JSON, pemetaan kanonik, korelasi warning, dan preservasi field tambahan.
P2	Worker per sumber, baca dari store, timeout, circuit breaker, batas konkurensi, rate limit, dan status stale.
P3	Kredensial per domain, JWT Ed25519, scope, proyeksi allowlist, dan rotasi refresh token.
P4	Container dan module terpisah, kepemilikan storage, serta PostgreSQL dengan atribut JSONB.
P5	Transactional outbox, Kafka, group consumer terpisah, SQLite persisten, dedup, dan DLQ.

2 Arsitektur Sistem

Gambar 1 memperlihatkan container, protokol, dan kepemilikan storage sistem.



Gambar 1: Arsitektur sistem beserta protokol dan kepemilikan penyimpanan. Setiap kotak dan simbol database mewakili satu container. Memori mock dan SQLite berada di dalam container pemilik. Panah dari Kafka menunjukkan aliran data menuju consumer yang menginisiasi fetch.

2.1 Alur kerja sistem

Alur sistem pada Gambar 1 terdiri atas penerimaan data, pelayanan permintaan client, dan distribusi event. Ketiganya menggunakan data kanonik yang sama, tetapi berjalan secara terpisah agar keterlambatan sumber atau consumer tidak langsung menahan permintaan baca.

Pada penerimaan data, worker Aggregator melakukan polling ke BMKG dan PVMBG dengan kredensial masing-masing. Record divalidasi, dipetakan menjadi HazardEvent, dan dikorelasikan dengan warning tsunami bila relevan. Data baru atau perubahan konten disimpan bersama snapshot outbox dalam transaksi PostgreSQL. Record yang isinya tetap tidak menghasilkan event baru. Checkpoint maju setelah seluruh respons selesai ditangani, sehingga kegagalan di tengah proses masih dapat dipulihkan melalui pembacaan ulang. Rincian pemetaan dan transaksi dibahas pada Bagian 5.

Pada pelayanan client, identitas ditukar menjadi token melalui Auth Service. Client kemudian mengirim permintaan ber-JWT ke Client API, yang memeriksa hak akses dan membatasi beban sebelum meneruskan query ke API internal Aggregator. Aggregator membaca Canonical Store tanpa memanggil sumber kembali. Client API memproyeksikan hasil sesuai scope dan menyertakan status kebaruan sumber. Pemisahan ini mempertahankan akses ke data tersimpan saat sumber bermasalah, dengan konsekuensi data dapat bersifat basi. Mekanismenya dijelaskan pada Bagian 6 dan 7.

Pada distribusi event, relay membaca outbox yang sudah committed dan memublikasikan snapshot ke Kafka. Setelah ACK broker diterima, baris outbox ditandai published. Dashboard Updater, Notifier, dan Pemda Portal memakai consumer group berbeda serta penyimpanan SQLite masing-masing. Dengan demikian, setiap consumer dapat memproses stream dan mengejar ketertinggalan secara independen tanpa meminta ulang data ke Aggregator. Batas pengiriman ulang dan idempotensi dibahas pada Bagian 9.

2.2 Komponen dan kepemilikan data

Sistem memiliki delapan service aplikasi apabila pemda-portal diaktifkan. PostgreSQL, Redis, dan Kafka merupakan container infrastruktur. Konfigurasi dasar menjalankan sepuluh container jangka panjang, ditambah satu `kafka-init` yang selesai setelah membuat topic; profile demo menambah pemda-portal. CLI dan load generator adalah client pengujian, bukan service penyimpanan data.

Komponen	Tanggung jawab	Storage dan port
<code>bmkg-mock</code>	Menyajikan gempa dan warning terpisah; seed dan generator; memvalidasi API key BMKG.	Memori sendiri; 8081.
<code>pvmbg-mock</code>	Laporan vulkanik, delay, perubahan skema, dan outage yang dapat dipicu operator.	Memori sendiri; 8082.
<code>aggregator</code>	Polling, tolerant reader, pemetaan, korelasi, transaksi, query internal, dan relay outbox.	Pemilik PostgreSQL; 9000 internal.
<code>client-api</code>	Verifikasi JWT, scope, proyeksi, filter, pagination, dan proteksi beban.	Tanpa store persisten; 8080.
<code>auth-service</code>	Penerbit access token dan refresh token dan pengelola rotasi sesi.	Pemilik Redis; 8090.

Komponen	Tanggung jawab	Storage dan port
dashboard-updater	View event terbaru per hazard, hanya menerima versi yang lebih baru.	SQLite sendiri; 8091.
notifier	Simulasi pengiriman alert SIAGA atau AWAS; dedup persisten.	SQLite sendiri; 8092.
pemda-portal	Subscriber tambahan untuk menunjukkan perluasan stream tanpa mengubah producer.	SQLite sendiri; 8093.

Port host aplikasi dipublikasikan pada loopback `127.0.0.1`. Port database, Redis, Kafka, dan API internal Aggregator tidak dipublikasikan ke host pada konfigurasi normal. Container memakai named volume untuk store persisten; restart biasa tidak menghapus state.

2.3 Protokol dan batas kepercayaan

Aggregator mengambil data JSON melalui HTTP dengan kredensial yang berbeda untuk setiap instansi. BMKG memerlukan `X-BMKG-Key`, sedangkan PVMBG memerlukan Bearer token miliknya sendiri. Client menukar kredensial ke `/oauth/token`, kemudian mengirim JWT ke client-api. Client-api membaca lewat HTTP internal Aggregator dengan `X-Internal-Key`; ia tidak mempunyai kredensial PostgreSQL. Auth-service mengakses Redis melalui RESP dan password yang berbeda.

Relay mengirim envelope melalui protokol Kafka. Consumer mengambil stream dengan group masing-masing dan melakukan efek lokal di SQLite. Tidak ada pemanggilan fungsi maupun shared business package lintas service. Kontrak yang sama direpresentasikan secara lokal di setiap module. Komponen dalam Aggregator tetap boleh berkomunikasi lewat port berupa interface Go karena berada pada service pemilik yang sama.

Network `store_net` menghubungkan Aggregator dan PostgreSQL; `edge_net` menghubungkan client-api dan Aggregator; `source_net` menghubungkan sumber dan poller; `auth_net` menghubungkan auth-service dan Redis; `bus_net` menghubungkan producer, broker, dan consumer. Profile test mempunyai akses yang diperlukan sebagai perangkat uji administratif, bukan jalur akses aplikasi produksi.

Implementasi terkait terdapat pada `docker-compose.yml`.

3 Teknologi, Asumsi, dan Batasan

3.1 Teknologi terpilih

Teknologi	Alasan terkait kebutuhan
Go 1.24.2	Goroutine dan context mendukung worker independen serta pembatasan waktu; module per service memisahkan dependensi build.
HTTP dengan JSON, net/http	Kontrak mudah diperiksa dan mendukung field aditif dari mock; biaya serialisasi diterima pada skala M1.
PostgreSQL 16.8, pgx	Transaksi menyatukan hazard dan outbox per record; checkpoint ditulis setelah respons selesai; kolom bertipe dipadukan dengan JSONB untuk atribut sumber.
Redis 7.4.2, AOF, Lua	TTL untuk sesi dan eksekusi rotasi atomik terhadap state Redis. Redis bukan penyimpanan private key.
JWT dengan EdDSA dan Ed25519	Client-api memverifikasi dengan public key tanpa panggilan auth-service pada setiap request.
Kafka 3.9.1, franz-go	Log persisten dan offset per group memungkinkan replay serta subscriber baru yang independen.
SQLite melalui modernc	View dan dedup lokal consumer tanpa menambah server database untuk setiap consumer.
Docker Compose	Container, network, volume, health check, dan konfigurasi tersusun dalam satu orkestrasi.
golang-migrate	Migrasi DDL berversi, di-embed dan dijalankan Aggregator; field aditif JSONB tidak memicu migrasi.
k6 0.57.0	Pengukuran beban berkelanjutan, percentile, throughput, dan error dengan denominator eksplisit.

3.2 Asumsi operasional dan data

Asumsi berikut merupakan prasyarat agar mekanisme sistem bekerja sesuai penjelasan laporan. Konsekuensi ketika prasyarat tidak terpenuhi dinyatakan bersama setiap asumsi.

1. Sistem dijalankan pada satu host Compose dengan satu instance per service, satu relay berurutan, dan satu broker Kafka. Penambahan penulis atau relay paralel memerlukan koordinasi tambahan agar pemrosesan tetap konsisten.
2. Jaringan demo dianggap lingkungan lokal tepercaya. HTTP internal tidak memakai TLS dan Kafka tidak memakai SASL. Isolasi network dan kredensial membatasi akses antaraplikasi, tetapi tidak melindungi kerahasiaan lalu lintas dari pihak yang menguasai host atau jaringan tersebut.
3. ID logis sumber tetap saat record yang sama dipoll ulang. Hazard disatukan melalui pasangan (**source**, **source_ref_id**). Stabilitas ID kanonik bergantung pada keberlangsungan Canonical Store.
4. Mock memakai seed minimal 20 record, generator default setiap 10 detik, dan memori lokal. Restart mock dapat kehilangan data runtime yang belum diambil; mekanisme ini bukan jaminan histori instansi produksi.
5. **since** gempa dan laporan bersifat inklusif menurut waktu kejadian atau laporan. Warning tidak memiliki timestamp perubahan pada kontrak wire; mock menyimpan waktu perubahan

secara internal untuk filtering. Kompatibilitas dengan mock lain memerlukan interpretasi `since` warning yang sama.

6. Watermark memakai waktu mulai request Aggregator dengan overlap 10 detik. Record baru atau perubahan harus masih tercakup oleh rentang `since` saat polling berikutnya. Selisih jam dan keterlambatan kemunculan data dapat membuat timestamp berada di luar rentang tersebut, sehingga record terlewat. Overlap bukan jaminan toleransi keterlambatan tak terbatas.
7. Tabel referensi gunung api tersedia secara statis dan sintetis. `volcano_id` yang tidak dikenal dikarantina, bukan diberi koordinat tebakan.
8. Jam Auth Service dan Client API cukup selaras untuk memeriksa waktu terbit dan kedaluwarsa JWT. Verifikator tidak menambahkan toleransi waktu, sehingga selisih jam dapat menyebabkan token sah ditolak. Validitas jam load generator juga diperlukan untuk pengukuran durasi, sebagaimana dibahas pada Bagian 6.

3.3 Batas layanan dan ketahanan

Sistem merupakan PoC lokal dan belum menyediakan high availability. Satu host, satu PostgreSQL, dan satu broker Kafka masih menjadi titik kegagalan. Named volume mempertahankan state saat container dimulai ulang, tetapi tidak menjamin pemulihan ketika seluruh volume hilang. Hasil load test lokal karena itu tidak menjadi klaim kapasitas atau ketersediaan deployment lain.

Jalur baca menyajikan data terakhir yang berhasil disimpan, bukan pembacaan sensor secara langsung. Status basi membantu client mengenali masalah sumber, tetapi tidak membuat data lama menjadi mutakhir. Evolusi skema dibatasi pada penambahan field JSON yang valid; perubahan field wajib atau arti data tetap memerlukan penyesuaian pemetaan.

Pengiriman event dapat berulang, dan urutan hanya terjaga dalam partisi Kafka dengan pemetaan key yang tetap. Consumer yang tertinggal melewati retensi tujuh hari tidak dijamin memperoleh seluruh histori. Notifier masih berupa simulasi log dan dapat mengirim ulang bila proses mati sesudah pengiriman tetapi sebelum marker persisten. Rincian batas ini terdapat pada Bagian 9.

Rotasi refresh atomik terhadap state Redis, tetapi tidak menjamin respons token baru sampai ke client. Pencabutan keluarga refresh juga tidak langsung membatalkan JWT yang belum kedaluwarsa. Konsekuensi terhadap sesi dan alasan pemilihan mekanisme dijelaskan pada Bagian 7.

3.4 Batas ukuran dan pemulihan backlog

Envelope event lengkap dibatasi hingga 4 MiB setelah serialisasi JSON UTF-8. Batas batch Kafka sebesar 5 MiB dan fetch sebesar 6 MiB menyediakan ruang bagi overhead protokol. Respons sumber dan halaman API dibatasi 8 MiB, sedangkan view consumer membatasi payload halaman hingga 7 MiB. Pagination mengikuti batas jumlah record maupun byte sehingga atribut event yang diterima tidak dipotong.

Event yang melebihi batas saat ingest dikarantina bersama payload sumber. Outbox lama yang ditolak permanen tetap disimpan dengan alasan penolakan dan tidak ditandai published. Relay dapat melanjutkan setelah penolakan tersimpan; akibatnya consumer dapat melihat lompatan versi snapshot. Pemulihan penolakan ini memerlukan operator. Respons sumber yang melampaui 8 MiB tetap menjadi keterbatasan karena kontrak sumber belum mempunyai pagination.

Timeout transaksi 2 detik berlaku per record. Checkpoint baru maju setelah semua record dalam respons selesai ditangani. Kegagalan di tengah tidak membatalkan prefix yang sudah

commit; pengulangan dengan hash mencegah outbox ganda. Kegagalan penyimpanan consumer mempertahankan offset untuk retry setelah restart worker, sedangkan DLQ digunakan bagi event yang tidak memenuhi kontrak.

4 Petunjuk Menjalankan Sistem

Prasyarat aplikasi adalah Go 1.24.2 dan Docker dengan Compose v2. Python 3 diperlukan untuk runner load dan helper trace. Perintah berikut dijalankan melalui PowerShell dari direktori utama repository.

```
powershell -NoProfile -File scripts/secrets/generate.ps1
docker compose config --quiet
docker compose up -d --build --wait --wait-timeout 180
```

Generator menyiapkan kredensial berbeda, pasangan key JWT, dan berkas environment lokal yang diabaikan Git. Nilai rahasia tidak disalin ke laporan. Pada POSIX, `make up` menyediakan bootstrap dan start. Pemda Portal dapat ditambahkan melalui perintah berikut.

```
docker compose --profile demo up -d --build pemda-portal
```

Pengujian unit dan analisis statis dengan `go vet` dijalankan melalui `scripts/check/check.ps1` atau `make check`. Pengujian runtime harus berurutan karena memicu perubahan skema, outage, dan restart dependensi. Rangkaian regresi lengkap dijalankan dengan perintah berikut.

```
$env:GOWORK='off'
go test ./scripts/check/foundation_test.go '
./scripts/check/ingest_test.go ./scripts/check/events_test.go '
./scripts/check/query_test.go -v -count=1 -timeout=15m
py scripts/loadtest/run.py
```

CLI dijalankan dari module `tools/field-cli`; kredensial dibaca dari environment lokal. Perintah berikut menjalankan sesi yang melewati masa berlaku access token.

```
go run ./cmd/field-cli list --raw --limit 1 --watch 65s --count 2
```

Gunakan identitas Tim Lapangan untuk raw. Endpoint `/health` menyatakan proses hidup; endpoint `/ready` aplikasi memeriksa dependensi yang relevan. Status proses hidup saja tidak membuktikan bahwa data semua sumber masih baru. Jejak satu alur dapat ditelusuri menggunakan correlation ID melalui perintah berikut.

```
py scripts/trace/trace.py <correlation-id>
```

Rincian endpoint, konfigurasi, cara memuat kredensial, dan skenario ada pada [README.md](#), [infra/README.md](#), [scripts/check/README.md](#), serta [scripts/loadtest/README.md](#). Penghentian dan pengaktifan kembali secara normal mempertahankan volume; menghapus volume tidak dipakai sebagai bukti recovery.

5 P1. Arsitektur dan Interoperabilitas Data

5.1 Masalah dan mekanisme

Kontrak inti sumber diketahui, tetapi field tambahan dapat muncul saat runtime. Karena itu parser memisahkan validasi field wajib dari penanganan field lain. Respons dibaca sebagai array `json.RawMessage`; setiap record diubah menjadi map nilai JSON. Field wajib diambil

dan diperiksa tipe dan rentangnya. Field yang tersisa menjadi **Extra**, kemudian diteruskan ke **attributes** tanpa dikonversi menjadi float64 secara umum.

Kebijakan yang dipilih adalah **mempertahankan field tak dikenal yang valid**. Ini mencakup **confidence_level**, objek bersarang, array, boolean, dan angka dalam rentang yang dapat disimpan JSONB. Nama sumber yang bertabrakan dengan atribut buatan BNPB, misalnya **tsunami_warnings**, ditempatkan pada namespace **source_extensions**. Poller mencatat nama field baru dalam log **schema_drift**. Field wajib yang hilang atau salah tipe, enum tak dikenal, referensi gunung yang tidak tersedia, atau nilai yang tidak dapat disimpan dikarantina dengan alasan yang eksplisit.

Record invalid dalam array tidak harus menggagalkan record lain. Akan tetapi, respons yang bukan JSON array valid menggagalkan fetch batch; watermark tidak dimajukan untuk fetch yang gagal. Penyimpanan karantina memakai pembungkus teks JSON mentah agar input yang tidak representabel sebagai JSONB masih dapat dicatat.

Implementasi terkait terdapat pada [services/aggregator/internal/application/canonicalize/input.go](#).

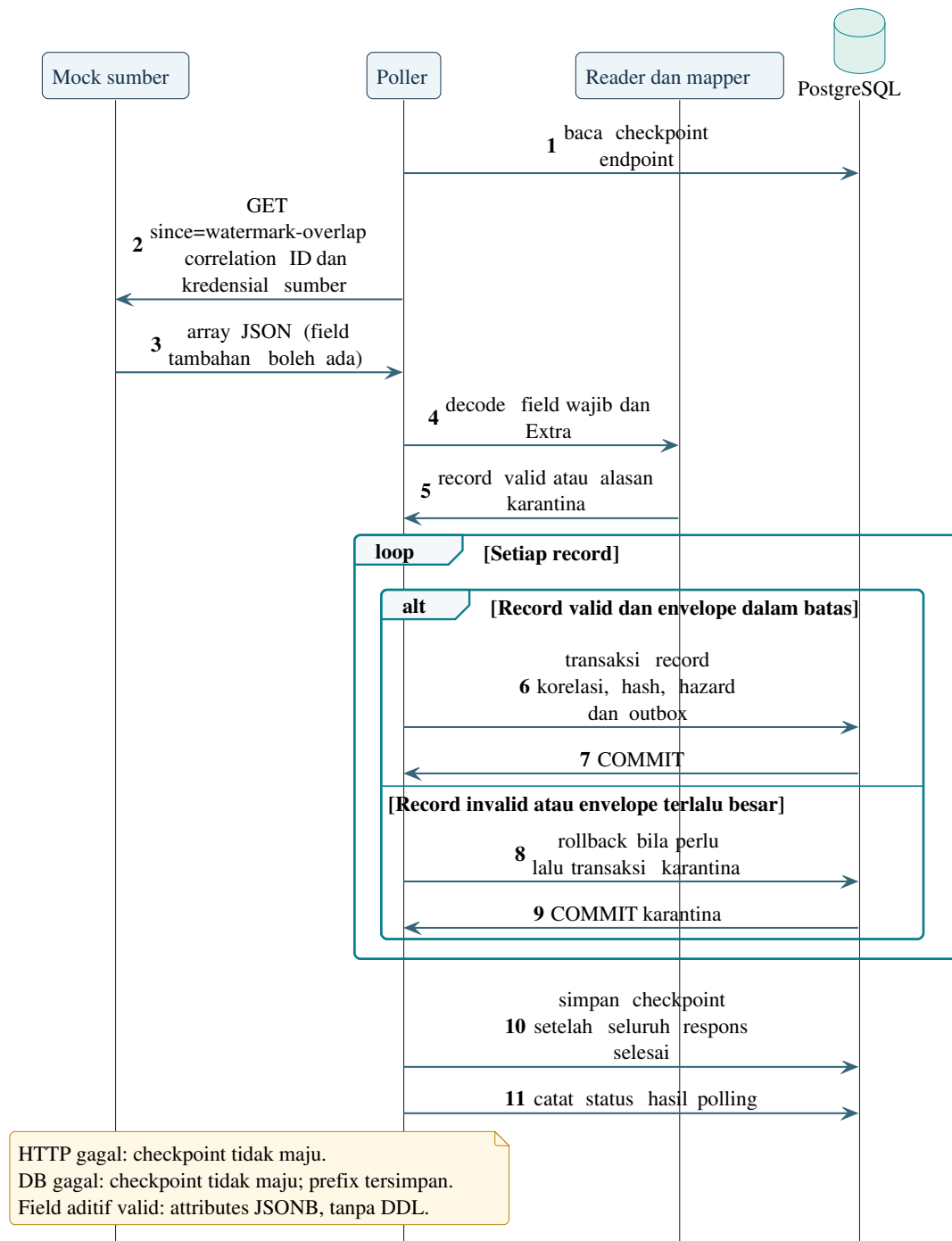
5.2 Pemetaan kanonik

Field dan aturan	BMKG	PVMBG
Identitas sumber	event_id menjadi source_ref_id .	report_id menjadi source_ref_id .
Jenis dan area	SEISMIC; region_name .	VOLCANIC; nama pada referensi volcano_id .
Lokasi dan waktu	Koordinat episentrum; occurred_at .	Koordinat referensi BNPB; reported_at .
Severity tanpa warning	NORMAL jika $M < 5$; WASPADA jika $5 \leq M < 6,5$; SIAGA jika $M \geq 6,5$.	Pemetaan langsung Normal, Waspada, Siaga, dan Awas ke enum huruf besar.
Severity dengan warning	Untuk gempa berpotensi tsunami, ikuti tingkat warning terkait; ambil tingkat tertinggi bila lebih dari satu.	Tidak berlaku.
Atribut sumber	Magnitude, kedalaman, potensi tsunami, warning terkait, serta field tambahan.	ID gunung, alert asli, jumlah erupsi, tinggi abu, serta field tambahan.

Aggregator membangkitkan UUID saat hazard pertama dibuat. Upsert mempertahankan identitas itu melalui unique constraint sumber dan ID sumber. **ingested_at** merekam penerimaan pertama, bukan waktu pembacaan terakhir; perubahan berikutnya dicatat dalam metadata internal seperti **updated_at** dan **version**.

Warning disimpan terpisah sehingga boleh mendahului gempa. Ketika gempa tiba, warning yang sudah ada digunakan; ketika warning datang belakangan, hazard terkait dikorelasikan ulang. **related_event_id** yang berubah untuk satu warning ditolak. Hasil warning dan zona diurutkan untuk menghindari perubahan hash hanya karena urutan array yang tidak bermakna.

Implementasi terkait terdapat pada [services/aggregator/internal/application/canonicalize/mapper.go](#).



Gambar 2: Ingest dan penambahan field sumber. Kotak transaksi mencakup state yang harus berubah bersama; tidak mencakup publish Kafka.

5.3 Deteksi perubahan dan transaksi

Hash konten dihitung dari makna field kanonik dan atribut; ID BNPB serta timestamp penerimaan tidak menjadi alasan perubahan baru. Angka dan timestamp dinormalisasi agar hasil stabil setelah round-trip penyimpanan. Record yang sama tetap memperbarui informasi pengamatan internal tanpa menghasilkan event baru. Perubahan konten meningkatkan version dan menghasilkan snapshot outbox dengan event ID baru.

Unit of work memakai transaksi PostgreSQL serializable per record. Upsert hazard, warning terkait, dan outbox berubah atomik. Checkpoint dimajukan setelah seluruh respons selesai, termasuk karantina record invalid. Jika record berikutnya gagal, bagian yang telah committed tetap tersimpan, sedangkan checkpoint lama menyebabkan replay yang aman melalui hash dan version. Batas transaksi per record mencegah backlog besar terus melampaui deadline transaksi tunggal.

Implementasi terkait terdapat pada `services/aggregator/internal/application/ingest/service.go`.

```
1 func MapVolcanic(v VolcanicInput, reference map[string]Volcano) (hazard.Event, error) {
2     r, ok := reference[v.VolcanoID]
3     if !ok {
4         return hazard.Event{}, errors.New("unknown volcano_id")
5     }
6     a := extras(v.Extra)
7     a["volcano_id"] = raw(v.VolcanoID)
8     a["alert_level"] = raw(v.Alert)
9     a["eruption_count_24h"] = raw(v.Eruptions)
10    a["ash_column_height_m"] = raw(v.Ash)
11    return hazard.Event{Source: "PVMBG", SourceRefID: v.ID, Type: "VOLCANIC", Severity: tsunami.
        Severity(v.Alert), Area: r.Name, Latitude: r.Latitude, Longitude: r.Longitude, OccurredAt: v.
        ReportedAt, Attributes: a}, nil
12 }
```

Listing 1: Pemilihan atribut tambahan pada pemetaan vulkanik; kutipan langsung kode implementasi.

5.4 Alternatif dan trade-off

Alternatif	Manfaat	Konsekuensi dan keputusan
Struct ketat dan tolak unknown field	Kontrak sangat eksplisit.	Penambahan aditif memaksa perubahan gateway; tidak dipilih.
JSON tolerant reader	Sesuai wire sumber dan mempertahankan field baru.	Perlu validasi runtime, kebijakan konflik nama, serta batas tipe; dipilih.
Schema registry dengan Avro atau Protobuf	Evolusi skema dapat dikelola dengan aturan formal.	Memerlukan pengelolaan kontrak tambahan dan tidak menggantikan kewajiban menerima wire JSON mock; ditunda.

Yang didukung ialah penambahan field valid tanpa mengubah field wajib. Rename, penghapusan field wajib, perubahan tipe atau arti field, dan enum baru tidak otomatis didukung. Mempertahankan JSON juga tidak berarti menafsirkan semantik field baru secara otomatis. `confidence_level` tersimpan sebagai atribut tambahan; laporan ini tidak mengklaim adanya model kepercayaan yang memakai nilainya untuk menghitung severity.

5.5 Bukti penyelesaian

`TestIngestPipeline` memicu schema v2 dan menemukan `confidence_level` pada Canonical Store serta log drift. `TestDynamicSchemaAndStaleAPI` memeriksa record skema lama dan baru melalui API, versi migrasi tidak berubah, serta container terkait tidak diganti. Pemetaan, field tambahan, korelasi, dan hash diperiksa pada unit test mapper. Rincian kriteria dan lokasi bukti terdapat pada Lampiran A.

Bukti eksekusi tersedia pada [docs/evidence/integration/regression.txt](#).

Implementasi terkait terdapat pada [services/aggregator/internal/application/canonicalize/mapper_test.go](#).

6 P2. Konkurensi, Availability, dan Degradasi

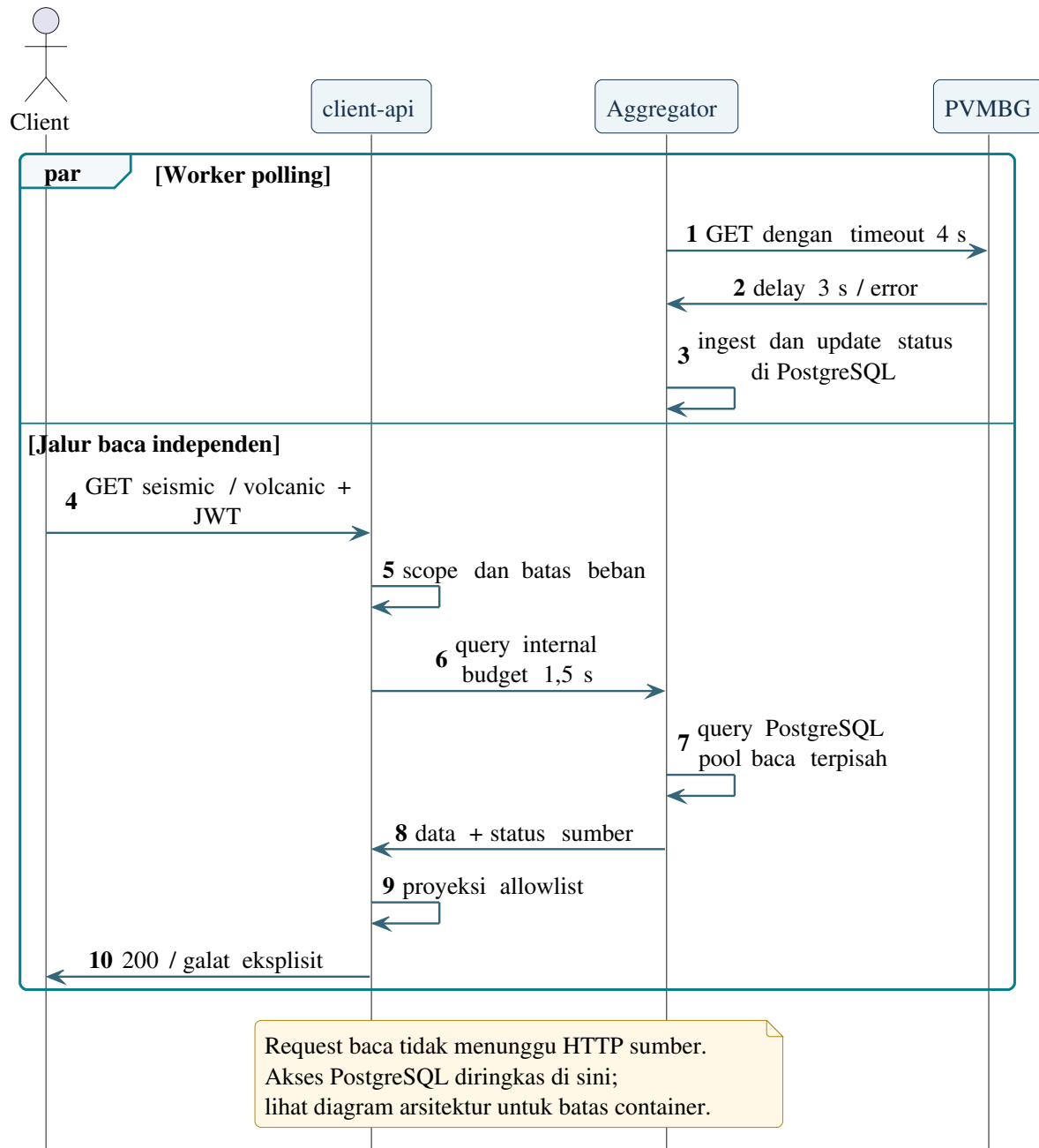
6.1 Model konkurensi dan isolasi

Go menjalankan worker BMKG dan PVMBG secara independen. Dua endpoint BMKG di-fetch paralel, lalu hasilnya diterapkan secara berurutan dalam worker BMKG. Worker tidak memulai siklus berikutnya sebelum siklus aktif selesai; konkurensi fetch tidak tumbuh tanpa batas. Tiap endpoint mempunyai state circuit breaker sendiri.

Jalur baca client mengambil data dari Canonical Store melalui query internal Aggregator, sehingga tidak ikut menunggu PVMBG. Pool PostgreSQL query dipisahkan dari pool ingest. Pembatasan pool membatasi kompetisi koneksi, tetapi keduanya masih memakai proses dan database yang sama. Pemakaian CPU, disk, dan lock database karena itu belum sepenuhnya terisolasi.

Cabang paralel pada Gambar 3 menunjukkan dua pekerjaan yang tidak saling menunggu. Timeout dan circuit breaker berlaku pada worker polling, sedangkan permintaan client mempunyai batas waktu query sendiri. Pemeriksaan scope dan batas beban dilakukan sebelum query agar permintaan yang tidak berhak atau melampaui kapasitas tidak memakai koneksi database. Proyeksi diterapkan setelah query untuk membentuk respons sesuai hak client. Ketika PVMBG gagal, jalur baca tetap dapat mengembalikan data tersimpan beserta status basi; independensi ini tidak berlaku jika Canonical Store sendiri tidak tersedia.

Kegagalan	Mekanisme dan batasnya
Head-of-line blocking	Worker sumber terpisah dan materialisasi data mengeluarkan HTTP sumber dari jalur request client. Read tidak menjamin data terbaru sensor.
Resource exhaustion	Semaphore request aktif, token bucket per identitas, pagination, pool koneksi terbatas, dan timeout. Penolakan terkendali dilaporkan sebagai 429, bukan disamarkan sebagai sukses.
Tidak ada degradasi	Status per endpoint diagregasi menjadi status sumber. Data lama tetap tersedia dengan <code>stale_since</code> ; sumber tanpa data usable dapat menghasilkan 503 eksplisit.



Gambar 3: Pemisahan request baca dan polling sumber. Pembacaan seismik tetap menggunakan store ketika panggilan PVMBG lambat atau gagal.

6.2 Konfigurasi dasar

Parameter	Nilai default yang dilaporkan
Interval polling BMKG dan PVMBG	BMKG 2 s dan PVMBG 5 s; jitter kurang dari 10% interval.
Overlap watermark	10 s per endpoint.
Timeout HTTP sumber BMKG dan PVMBG	BMKG 1 s dan PVMBG 4 s.
Timeout operasi dan transaksi PostgreSQL	2 s; deadline parent yang lebih pendek tetap berlaku.
Timeout client-api ke Aggregator	Maksimum 1,5 s total.
Breaker	Buka setelah 3 kegagalan fetch; cooldown 10 s.
Pool PostgreSQL ingest dan query	Ingest 5 koneksi dan query 3 koneksi.
Request aktif client-api	Maksimum 100.
Token bucket per identitas	100 request per detik; kapasitas burst 200.
Ukuran halaman	Default 100, maksimum 500.

Implementasi terkait terdapat pada [services/aggregator/internal/config/config.go](#).

Implementasi terkait terdapat pada [services/client-api/internal/config/config.go](#).

Breaker menahan fetch baru selama sumber berulang kali gagal. Poll yang dilewati bukan percobaan HTTP baru. Saat cooldown selesai, percobaan yang berhasil memulihkan alur tanpa restart. Status sumber sehat baru dapat diketahui setelah polling dan pembaruan status berikutnya; mematikan outage tidak membuat metadata langsung HEALTHY pada detik yang sama.

6.3 Kebaruan data dan kontrak galat

stale_since menunjukkan awal masalah yang terdeteksi, bukan waktu persis instansi mulai mati. Pada kondisi normal, perkiraan lag mencakup waktu menunggu polling, jitter, durasi fetch, dan transaksi penyimpanan. Secara konseptual,

$$L_{\text{ingest}} \approx W_{\text{poll}} + J + T_{\text{HTTP}} + T_{\text{tx}}.$$

Perkiraan tersebut tidak menjadi jaminan batas waktu layanan. Siklus yang panjang, kegagalan database, ketidakselarasan jam, atau circuit breaker dapat menambah waktu ingest. Outage berkepanjangan membuat umur data terus bertambah. Untuk jalur consumer, tambahkan waktu tunggu relay, ACK broker, dan pemrosesan consumer. Klaim “maksimal satu interval polling” tidak digunakan.

Filter tidak valid menghasilkan 400, identitas atau token invalid 401, akses field terlarang 403, detail yang tidak ditemukan 404, beban ditolak 429, dan dependensi tidak tersedia 503. Hasil filter kosong yang sah tetap 200 dengan array kosong. Error tidak membocorkan detail SQL atau secret.

Metadata **sources** tersedia pada list serta detail biasa dan raw. Metadata tetap disertakan ketika client memilih sebagian field hazard, sehingga pembaca detail dapat mengetahui status basi. Instrumentasi outbound mencatat latency dan correlation ID untuk HTTP sumber, PostgreSQL, Redis, serta Kafka tanpa menulis SQL, argumen perintah, atau secret. Permintaan protokol Kafka yang mencakup beberapa record memakai identitas operasi tersendiri; publish dan commit per record mempertahankan correlation ID event.

6.4 Konfigurasi dan hasil pengukuran

Pengukuran terbaru dilakukan pada 8 Oktober 2026 terhadap revisi **a465139** menggunakan k6 0.57.0 native Windows. Docker Desktop 28.0.4 menyediakan 16 CPU dan 7.936.241.664 byte RAM untuk lingkungan container. Tool load dibangun dengan Go 1.23.6; service memakai Go 1.24.2. Angka di bagian ini diambil dari artefak JSON pengujian ulang.

PVMBG diatur dengan delay minimum dan maksimum 3 s. Skenario paralel menjalankan 10 VU seismic dan 10 VU volcanic selama 60 s. Skenario sustained menjalankan 50 VU selama 90 s dengan jeda 0,1 s per iterasi; setiap VU memiliki sesi refresh sendiri. Ini model closed loop. Semua VU menggunakan identitas Tim Lapangan yang sama, sehingga berbagi rate limit client. Skenario outage memakai 5 VU per fase (20 s outage, 30 s recovery) dan satu VU kontrol.

Skenario	Bisnis/s	Sukses/s	p50 (ms)	p95 (ms)	p99 (ms)
seismic-only	188,49	99,72	6,63	12,27	18,67
sustained	471,80	97,59	6,89	13,93	27,11
outage	32,47	32,47	4,56	6,82	7,91

Tabel 2: Pengujian ulang k6 pada 8 Oktober 2026. Latency hanya HTTP 200; bisnis/s mencakup 429.

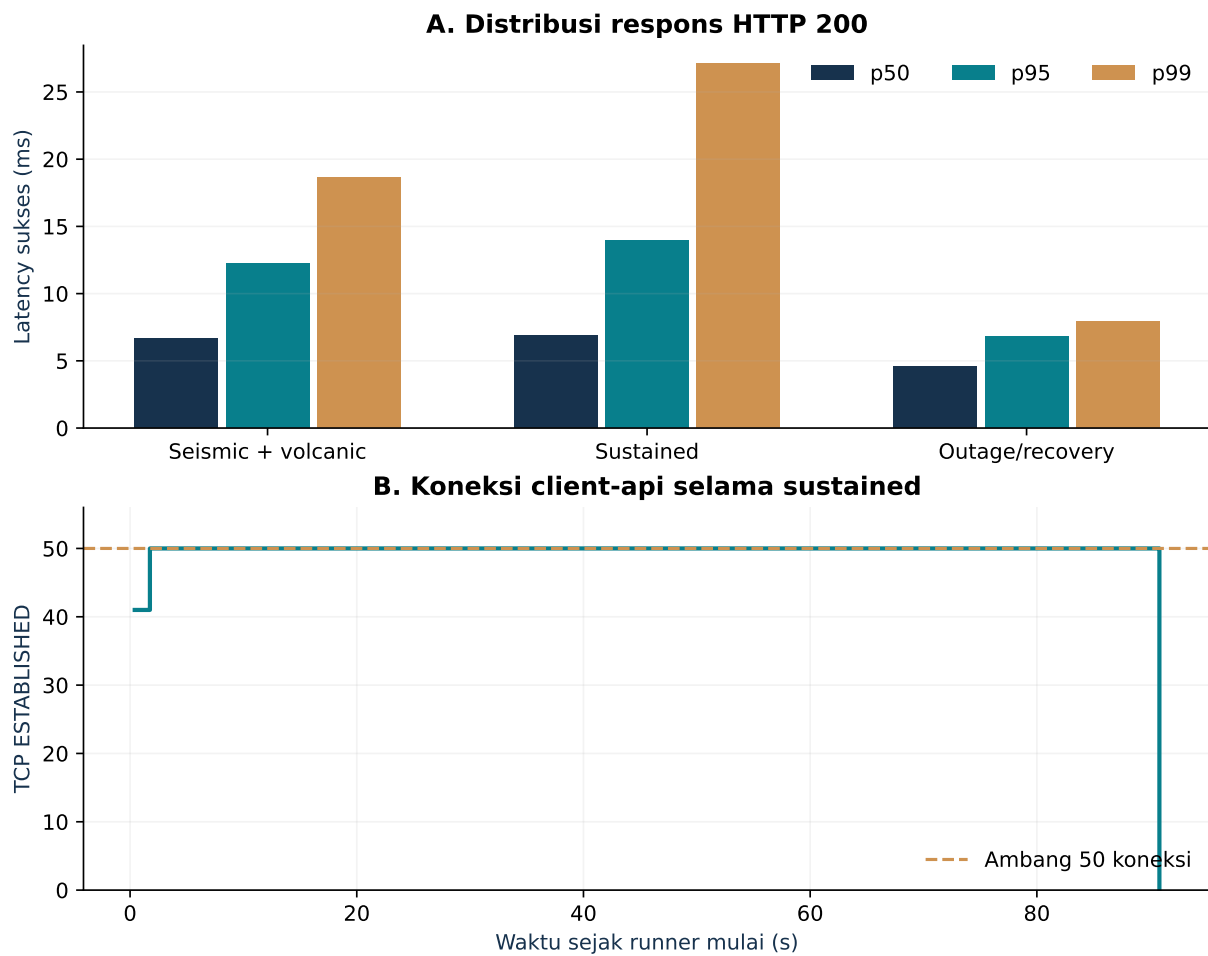
Percentile dalam tabel adalah respons HTTP 200 gabungan masing-masing skenario. Untuk kriteria khusus BMKG-only, p95 **seismic saja 12,51 ms**, lebih kecil dari 300 ms ketika pembacaan volcanic berlangsung bersamaan. Durasi mengikuti definisi k6 yang mencakup waktu pengiriman, penantian respons, dan penerimaan data, tetapi tidak mencakup pembentukan koneksi [7]. Penolakan 429 tidak dimasukkan ke distribusi latency sukses.

Pada sustained, 42.511 respons bisnis terdiri atas 8.793 HTTP 200 dan 33.718 HTTP 429. Ada 9 penolakan autentikasi 429 yang dicatat terpisah; auth error setelah mekanisme retry 0%. Throughput sebesar 471,80 respons bisnis per detik mencakup penolakan akibat pembatasan beban. Throughput HTTP 200 sebesar 97,59 respons per detik konsisten dengan batas 100 request per detik untuk satu identitas yang dipakai seluruh VU. Sebanyak 79,32% request bisnis ditolak secara terkontrol. Hasil ini menunjukkan kestabilan pada pola tersebut, tetapi belum mengukur kapasitas maksimum server.

Error bisnis dihitung sebagai jumlah respons gagal di luar penolakan terkontrol dibagi jumlah respons bisnis selain 429. Pada sustained hasilnya $0/8793 = 0\%$. Sampel 429 dikeluarkan dari pembilang *dan* penyebut agar tidak mengencerkan error rate. Total HTTP bawaan k6 mencampurkan auth, control, dan retry, sehingga bukan metrik throughput bisnis utama.

Runner membaca `/proc/net/tcp` dan `/proc/net/tcp6` sekitar setiap satu detik untuk menghitung koneksi TCP ESTABLISHED pada port client-api. Sebanyak 67 sampel menunjukkan sedikitnya 50 koneksi teramati terus-menerus selama **87,78 s**. Koneksi aktif secara TCP tidak berarti semua request sedang diproses serentak; jumlah VU saja juga tidak dipakai sebagai bukti koneksi.

Selama outage singkat, seluruh 480 respons seismic tetap sukses dengan data dan status BMKG HEALTHY. Sebanyak 435 dari 480 respons volcanic menampilkan data tersimpan dengan penanda basi; 45 respons awal masih berada sebelum masalah sumber terdeteksi. Pada fase recovery, 625 dari 730 respons (85,62%) sudah kembali HEALTHY; 105 respons lainnya masih berada dalam masa pemulihan, tetapi tetap HTTP 200. Pemulihan teramati pada fase recovery sebelum runner mengembalikan konfigurasi mock dan tidak memerlukan restart aplikasi. Hasil ini memenuhi threshold internal script di atas 50%, tetapi bukan jaminan waktu pemulihan seketika.



Gambar 4: Grafik yang dibangkitkan langsung dari JSON hasil k6 dan sampling koneksi. Percentile memakai respons sukses; koneksi diukur terpisah dari jumlah VU.

Metrik dan konfigurasi lengkap tersimpan pada [docs/evidence/final-2026-10-08/README.md](#). Angka berasal dari [docs/evidence/final-2026-10-08/load/seismic-only.json](#), [docs/evidence/final-2026-10-08/load/sustained.json](#), [docs/evidence/final-2026-10-08/load/connections.json](#), serta [docs/evidence/final-2026-10-08/load/outage.json](#).

6.5 Outage berkepanjangan

Pengujian tambahan menonaktifkan PVMBG selama 600 detik dan mengamati recovery selama 90 detik. Lima VU membaca seismic serta volcanic selama outage; lima VU membaca volcanic pada fase recovery. Satu pemeriksa terpisah mengambil sampel status sekitar setiap lima detik. Konfigurasi dan hasil lengkap tersedia pada [docs/evidence/final-2026-10-08/long-outage/configuration.json](#) dan [docs/evidence/final-2026-10-08/long-outage/summary.json](#).

Seluruh 14.281 pembacaan seismic selama outage tetap sukses dengan sumber BMKG HEALTHY. Sebanyak 14.226 dari 14.281 pembacaan volcanic menampilkan status basi; 55 pembacaan awal terjadi sebelum gangguan terdeteksi. Seluruh 30.753 respons bisnis pada kedua fase adalah HTTP 200, tanpa penolakan 429 maupun error bisnis. Pada recovery, 2.151 dari 2.191 respons volcanic sudah HEALTHY. Sampel pertama yang memperlihatkan pemulihan tercatat sekitar detik ke-605,31 sejak runner dimulai. Waktu ini merupakan observasi dengan interval sampling, bukan jaminan batas waktu pemulihan.

Sebanyak 136 sampel berkala mendukung pemeriksaan status sumber, dengan 117 sampel saat outage aktif. Identitas container, waktu mulai, dan jumlah restart sebelum serta sesudah pengujian tetap sama. Dengan demikian, pemulihan berlangsung tanpa restart atau penggantian container. Bukti pemeriksaan tersedia pada [docs/evidence/final-2026-10-08/long-outage/result.json](#) dan [docs/evidence/final-2026-10-08/long-outage/samples.jsonl](#).

6.6 Validitas hasil dan alternatif

Percobaan awal k6 di container menghasilkan durasi negatif, misalnya minimum -541,10 ms. Walaupun threshold k6 saat itu tidak gagal, hasil tersebut tidak digunakan untuk klaim performa. Artefaknya tetap disimpan pada [docs/evidence/integration/load-docker-invalid/seismic-only.json](#). Penyebab lingkungan belum diisolasi secara pasti. Runner kini menolak nilai durasi negatif, dan seluruh pengukuran final diulang dengan k6 native Windows terhadap stack yang sama.

Alternatif request-time fan-out menjaga kemungkinan kebaruan lebih tinggi, tetapi menempatkan latensi dan kegagalan sumber di jalur client. Materialisasi dipilih karena data historis masih berguna dan P2 menuntut degradasi terkendali. Cache dan micro-cache dapat mengurangi beban query, tetapi menambah invalidasi dan state; belum digunakan. Banyak instance dengan load balancer dapat meningkatkan kapasitas, tetapi memerlukan limit terdistribusi dan koordinasi penulis yang berada di luar M1. Hasil satu mesin ini tidak digeneralisasi sebagai kapasitas produksi.

7 P3. Autentikasi dan Domain Kepercayaan

7.1 Pemisahan kredensial

Domain	Mekanisme	Pemisahan yang ditegakkan
BMKG	Header X-BMKG-Key .	Hanya API key BMKG yang sah.
PVMBG	Authorization: Bearer token PVMBG.	Nilai berbeda dari key BMKG dan JWT downstream.
Admin simulasi	X-Admin-Key .	Kredensial operator terpisah dari akses data.
Downstream	Client ID dan secret berbeda untuk Media, Lapangan, BNPB.	Auth-service menerbitkan scope berdasarkan konfigurasi server.
API internal	X-Internal-Key .	Client-api mengakses Aggregator; bukan token publik client.
Penyimpanan	Password PostgreSQL dan Redis terpisah.	Hanya service pemilik menerima konfigurasi store.

Secret dibangkitkan oleh bootstrap dan dibaca dari environment atau file lokal; private key hanya dipasang pada auth-service dan public key dipasang pada client-api. Berkas contoh memuat placeholder. Mengetahui kredensial Media tidak memberikan kredensial sumber ataupun private key penerbit JWT. Pemisahan ini mempersempit dampak kebocoran satu domain, tetapi bukan pengganti enkripsi transport pada deployment nyata.

7.2 JWT, scope, dan proyeksi

Access token ditandatangani EdDSA dengan key Ed25519 dan TTL default 60 detik. Verifikator membatasi algoritma yang diterima serta memeriksa issuer, audience, expiration, issued-at, subject, ID token, dan scope. Client-api memakai public key lokal, sehingga verifikasi tidak membutuhkan panggilan auth-service untuk setiap request. Refresh token berupa nilai acak opaque 32 byte; Redis menyimpan hash dan metadata sesi.

Identitas	Hak akses
Media	Ringkasan yang mencakup hazard ID, sumber, jenis, severity, nama area, occurred-at, dan ingested-at.
Tim Lapangan	Ringkasan beserta data mentah yang mencakup source reference ID, koordinat, dan seluruh attributes.
BNPB Pusat	Ringkasan dan mentah; baseline hak tertinggi pada M1 yang tetap read-only.

Permintaan eksplisit field mentah oleh Media ditolak server dengan 403, termasuk raw detail dan **include=raw**. Sesudah query, projector membangun objek baru berdasarkan allowlist. Menambah field upstream dengan demikian tidak otomatis membocorkannya ke Media. Menyembunyikan koordinat hanya pada UI tidak dipakai sebagai mekanisme keamanan.

Implementasi terkait terdapat pada [services/client-api/internal/authn/verifier.go](https://github.com/AkuAnakTehat/services/blob/main/client-api/internal/authn/verifier.go).

Implementasi terkait terdapat pada [services/client-api/internal/projection/projector.go](https://github.com/AkuAnakTehat/services/blob/main/client-api/internal/projection/projector.go).

```

1 func Project(h map[string]any, scope string, fields []string) map[string]any {
2     allowed := append([]string{}, authz.Summary...)
3     if authz.Has(scope, "hazard:read:raw") {
4         allowed = append(allowed, authz.Raw...)
5     }
6     selected := map[string]bool{}
7     for _, f := range fields {
8         selected[f] = true
9     }
10    result := make(map[string]any)
11    for _, f := range allowed {
12        if len(fields) > 0 && !selected[f] {
13            continue
14        }
15        if value, ok := h[f]; ok {
16            result[f] = value
17        }
18    }
19    return result
20 }

```

Listing 2: Proyeksi membangun objek baru dari allowlist, bukan menghapus beberapa field dari objek mentah.

7.3 Rotasi refresh dan batas atomisitas

Gambar 5 menunjukkan penerbitan sesi awal melalui kredensial Tim Lapangan, dilanjutkan pembaruan sesi memakai refresh token. Access token yang singkat membatasi masa penggunaan token yang bocor, sedangkan refresh token memungkinkan CLI memperoleh access token baru tanpa meminta pengguna memasukkan kredensial pada setiap pembaruan. Validasi dan penggantian refresh dilakukan dalam satu operasi Redis agar dua permintaan tidak sama-sama berhasil memakai token lama.

Redis menyimpan keluarga sesi dengan expiry tetap. Lua memvalidasi tipe key, keberadaan, family aktif, kecocokan metadata, dan TTL sebelum mutasi. Pada rotasi normal, refresh lama ditandai used dan refresh baru dibuat dengan sisa masa berlaku yang tidak memperpanjang keluarga. Pemakaian kembali refresh lama mencabut keluarga. Semua langkah tersebut atomik terhadap pengamatan state Redis; script juga menyusun validasi sebelum mutasi karena error runtime Lua tidak otomatis melakukan rollback.

Atomisitas Redis tidak mencakup respons HTTP. Jika rotasi berhasil tetapi respons token baru hilang, retry token lama dapat dianggap reuse; client mungkin harus login ulang. Karena itu satu refresh token tidak digunakan secara paralel oleh CLI maupun antar-VU k6. Ini trade-off proteksi replay yang ketat terhadap kelangsungan sesi pada kegagalan jaringan tertentu.

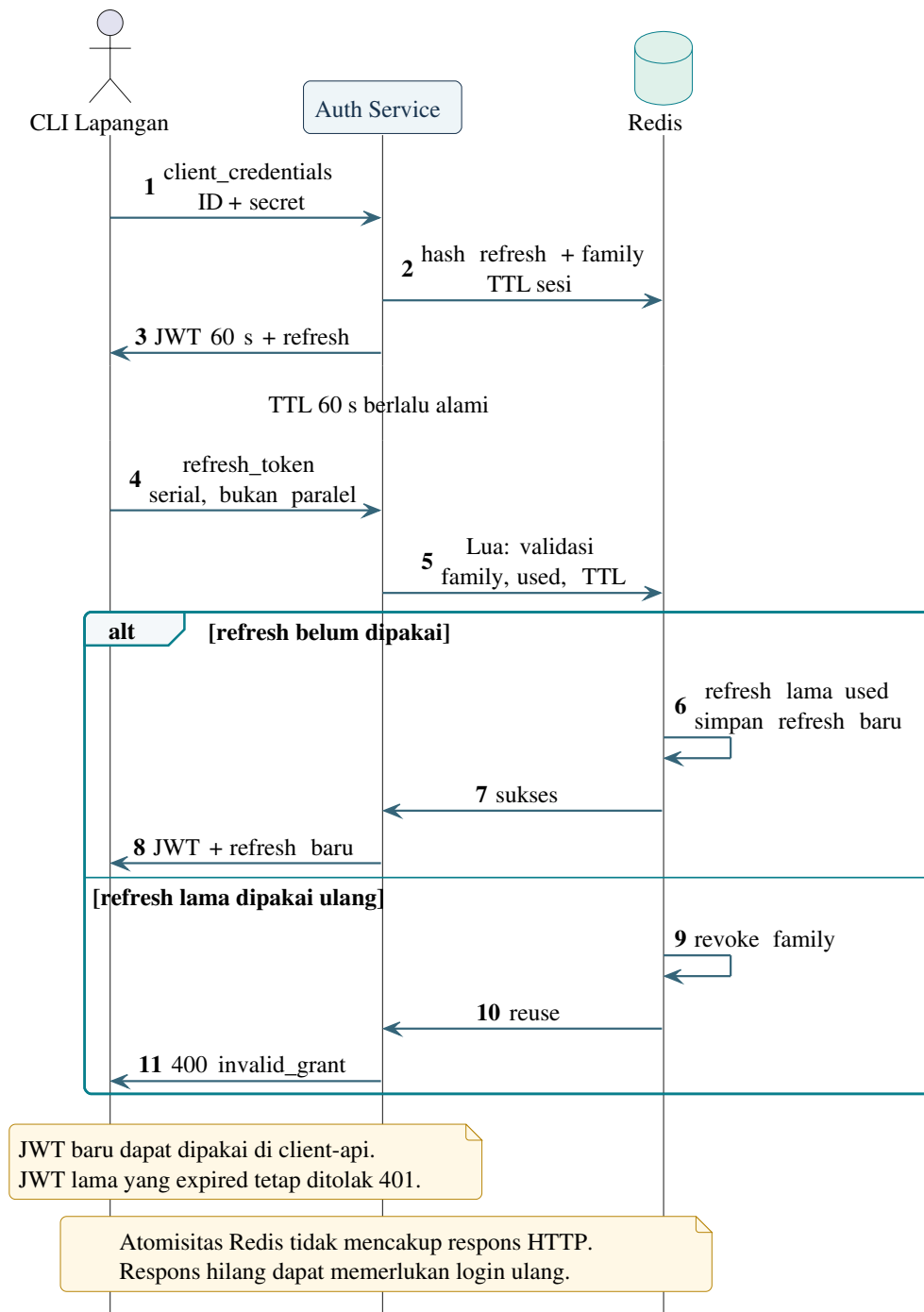
JWT yang sudah diterbitkan diverifikasi offline. Pencabutan refresh family tidak langsung mencabut access token yang masih belum expired. Jendela itu dibatasi TTL pendek; mekanisme introspection atau denylist access token belum diterapkan. Dalam demo expiry, token lama ditolak karena waktu kedaluwarsanya telah lewat, bukan karena refresh secara ajaib membatalkan semua access token lama.

Implementasi terkait terdapat pada [services/auth-service/internal/adapter/outbound/redis/rotate.lua](#).

Implementasi terkait terdapat pada [tools/field-cli/internal/client/client.go](#).

7.4 Alasan pemilihan dan risiko yang ditangani

Satu kredensial bersama antarinstitusi ditolak karena menghapus otonomi trust. Session lookup terpusat untuk setiap request menyederhanakan revocation langsung, tetapi menjadikan auth-



Gambar 5: Sesi Tim Lapangan, rotasi refresh, serta pemeriksaan token access lama yang sudah kedaluwarsa.

store dependensi jalur baca. JWT asimetris dipilih agar client-api hanya memiliki kemampuan verifikasi; berbeda dari shared signing secret yang akan memperluas kemampuan penerbitan token.

Validasi algoritma dan claim menghindari penerimaan token untuk issuer atau audience yang salah. Allowlist dan pemeriksaan scope berada di server. Help CLI tidak mencetak secret dari environment; redirect HTTP ditolak untuk mencegah penerusan body login ke tujuan lain. Galat dan trace tidak memuat token atau password. Implementasi ini masih ditujukan untuk PoC lokal. TLS dan mTLS, rotasi key online, MFA, serta pencabutan access token secara langsung belum diimplementasikan. Oleh sebab itu tidak ada klaim bahwa sistem bebas dari seluruh celah keamanan.

7.5 Bukti penyelesaian

Suite fondasi memeriksa kredensial lintas instansi ditolak serta claim JWT invalid tidak diterima. Suite query memverifikasi allowlist Media, penolakan raw, dan akses penuh dua identitas lainnya. `TestNaturalExpiryAndFieldCLI` menjalankan dua request CLI berjarak 65 detik dalam satu sesi dengan default TTL 60 detik, lalu memeriksa token lama 401 sebelum dan sesudah refresh dan token baru 200. Tidak ada manipulasi jam atau TTL singkat buatan untuk bukti ini.

Uji unit CLI memeriksa rotasi serial, respons 401 yang terlambat tidak membatalkan token yang lebih baru, serta penolakan redirect. Pada pemeriksaan final 9 Oktober 2026, Gitleaks 8.24.2 memindai 58 commit hingga 48987b7 tanpa temuan. Bukti final tersedia pada `docs/evidence/reliability-2026-10-09/secret-scan.txt`; pemeriksaan 8 Oktober hingga 2353479 tetap menjadi bukti historis sebelumnya. Semua nilai contoh pada laporan bebas kredensial nyata.

Bukti eksekusi tersedia pada `docs/evidence/integration/regression.txt`.

Bukti eksekusi tersedia pada `docs/evidence/integration/modules.txt`.

Implementasi terkait terdapat pada `docs/evidence/foundation/README.md`.

8 P4. Batas Service dan Flexible Storage

8.1 Alasan menentukan batas service

BMKG dan PVMBG dipisah karena mewakili instansi otonom, kontrak berbeda, serta kegagalan dan kredensial terpisah. Auth-service menjadi pemilik kebijakan penerbitan token dan sesi; client-api menjadi batas akses downstream. Aggregator menjadi pemilik data kanonik, pemetaan, dan transaksi perubahan data. Consumer dipisahkan karena mempunyai tujuan, siklus hidup, serta state lokal yang berbeda.

Poller, mapper, query internal, dan relay outbox tetap merupakan komponen dalam Aggregator. Memecah setiap komponen itu menjadi service baru akan menambah kontrak jaringan serta mempersulit kepemilikan transaksi tanpa kebutuhan skala M1 yang terbukti. Sebaliknya, menyatukan auth, penyajian client, dan ingest dalam satu deploy akan mengikat perubahan kebijakan akses pada perubahan pemetaan sumber. Pilihan saat ini menerima kompleksitas operasi beberapa container untuk memperoleh perubahan dan recovery yang lebih mandiri.

Setiap service memiliki module Go, entrypoint, Dockerfile, konfigurasi, dan endpoint health. Dependensi shared business package tidak digunakan. Tiga consumer memang memiliki sebagian kode kontrak dan mekanisme delivery serupa; pengulangan kecil ini menjaga build independen pada M1, dengan konsekuensi perubahan envelope perlu diselaraskan dan diuji pada setiap pemakai.

8.2 Kepemilikan Canonical Store

Client-api memanggil `/internal/hazards` dan detail melalui HTTP Aggregator. Ia tidak berada di `store_net` dan tidak menerima kredensial PostgreSQL. Consumer membaca Kafka dan menyimpan view lokal; auth-service memakai Redis. Dengan demikian tidak ada jalur aplikasi biasa yang membaca tabel kanonik secara langsung dari service lain.

Pembatasan ini berlaku pada konfigurasi aplikasi. Test PostgreSQL dan operator yang memiliki Docker dapat mengakses database untuk memeriksa bukti; hak administratif itu tidak diklaim dicegah oleh pemisahan network. Network bukan boundary terhadap administrator mesin.

Implementasi terkait terdapat pada `docker-compose.yml`.

Implementasi terkait terdapat pada `services/client-api/internal/adapters/outbound/aggregator/client.go`.

8.3 Skema dan evolusi atribut

Tabel	Fungsi
<code>hazard_events</code>	Field kanonik bertipe, <code>attributes</code> JSONB, keunikan pasangan sumber dan ID sumber, version dan content hash.
<code>outbox</code>	Snapshot perubahan, event ID unik, hazard dan version, dan tanda publish.
<code>checkpoints</code>	Watermark per endpoint yang committed bersama batch.
<code>tsunami_warnings</code>	Korelasi warning sebelum dan sesudah gempa.
<code>source_status</code> dan <code>source_endpoint_status</code>	Status sehat, kegagalan, waktu pengamatan, dan stale.
<code>quarantine</code>	Record yang ditolak beserta alasan dan trace.
<code>schema_migrations</code>	Versi DDL yang dikelola migration runner.

PostgreSQL menyimpan field kanonik yang sering difilter sebagai kolom dengan constraint dan indeks. Atribut yang tidak mempunyai padanan kanonik disimpan sebagai JSONB. Penambahan `confidence_level` dengan demikian merupakan perubahan isi dokumen pada kolom yang sudah ada, bukan penambahan kolom. Migrasi `002_source_status_stale_since` adalah perubahan struktur fitur status sumber, bukan langkah yang dilakukan saat demo penambahan confidence.

JSONB menyediakan representasi JSON yang dapat diproses dan diindeks, tetapi tidak mempertahankan whitespace, urutan key, atau duplicate keys seperti teks asli [5]. Maka “preservasi atribut” berarti mempertahankan nilai yang didukung, bukan salinan byte-for-byte seluruh wire payload. Indeks utama query saat ini mengikuti waktu kejadian dan hazard ID; tidak ada klaim semua key JSON dinamis otomatis terindeks.

Implementasi terkait terdapat pada `services/aggregator/migrations/001_initial.up.sql`.

Implementasi terkait terdapat pada `services/aggregator/migrations/002_source_status_stale_since.up.sql`.

8.4 Perbandingan alternatif penyimpanan

Pilihan	Keunggulan	Konsekuensi
PostgreSQL dengan JSONB (dipilih)	Transaksi hazard, outbox, dan checkpoint; constraint dan query kolom; atribut fleksibel.	Tetap membutuhkan pengelolaan DDL untuk field inti dan kapasitas DB; validasi atribut menjadi tanggung jawab aplikasi.
MongoDB sebagai document store	Dokumen bersarang dan field aditif cocok untuk data heterogen.	Konsistensi lintas record, outbox, indeks, dan operasi deployment tetap harus dirancang; fleksibilitas dokumen tidak otomatis menyelesaikan transaksi.
Relasional kolom tetap saja	Skema dan tipe seluruh atribut sangat eksplisit.	Field sumber baru memerlukan DDL dan koordinasi deploy; bertentangan dengan tujuan perubahan aditif saat runtime.

Perbandingan ini merupakan pertimbangan desain, bukan benchmark database. JSONB dipilih karena kebutuhan transaksi dan query kanonik sudah sesuai dengan PostgreSQL; mengganti store tidak memberikan manfaat M1 yang cukup untuk menambah model konsistensi baru.

8.5 Bukti independensi dan batas

`TestIndependentRebuild` mencatat timestamp start Aggregator, client-api, auth-service, kedua mock, dashboard-updater, dan Kafka; menghentikan notifier; memastikan baca Media masih berhasil; lalu build dan start notifier dengan `-no-deps`. Timestamp komponen lain tidak berubah. Ini bukti konkret satu skenario rebuild independen, bukan klaim semua kombinasi deployment sudah diuji.

`TestDynamicSchemaAndStaleAPI` mengambil record tanpa confidence, mengaktifkan skema baru, membaca record lama dan baru, serta memeriksa versi migrasi dan container tetap. Named volume Canonical Store dan SQLite mempertahankan data pada restart normal. SQLite cocok untuk satu penulis lokal; banyak replica consumer yang berbagi berkas SQLite atau multi-instance Aggregator belum didukung oleh desain koordinasi M1.

Bukti eksekusi tersedia pada [docs/evidence/integration/regression.txt](#).

Implementasi terkait terdapat pada [scripts/check/query_test.go](#).

9 P5. Distribusi Event dengan Consumer Independen

9.1 Broker dan pola yang dipilih

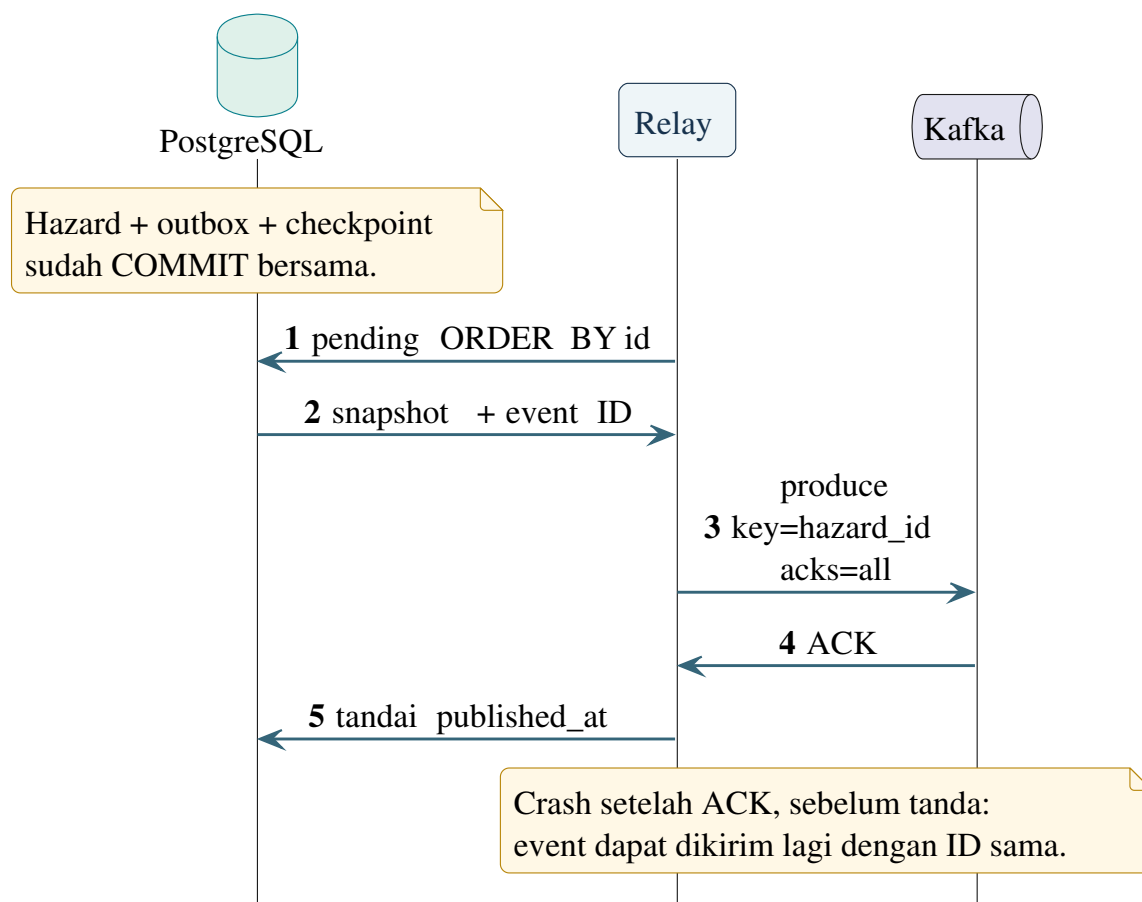
Kafka menyediakan log yang dapat dibaca ulang dan posisi baca per consumer group. Topic utama adalah `bnpb.hazard-events.v1`; kegagalan pesan masuk ke topic DLQ terpisah. Konfigurasi M1 memakai satu partisi, replication factor 1, serta retensi broker tujuh hari. Dashboard-updater, notifier, dan pemda-portal memakai group berbeda sehingga masing-masing memperoleh stream yang sama. Jika mereka memakai satu group, Kafka justru membagi pekerjaan antaranggota; perilaku itu tidak memenuhi kebutuhan tiga proyeksi independen.

Producer memakai key `hazard_id`. Envelope memuat schema version, event ID, event type, hazard ID, version, correlation ID, timestamp, dan snapshot hazard. Bentuk snapshot membuat consumer tidak perlu query balik ke Aggregator saat menangani event. Nama field `published_at` dalam envelope diisi ketika envelope dibuat pada transaksi ingest; waktu ACK publish dicatat pada kolom outbox. Keduanya tidak boleh dianggap timestamp yang identik.

Implementasi terkait terdapat pada [docs/api/hazard-event.md](#).

Implementasi terkait terdapat pada [infra/kafka/init-topics.sh](#).

9.2 Outbox dan semantik pengiriman



Gambar 6: Batas transaksi dan publish. ACK broker mendahului penandaan published pada PostgreSQL; crash di antaranya masih memungkinkan duplikasi.

Menulis database lalu publish langsung dapat kehilangan event ketika proses mati di antaranya. Memublikasikan lebih dahulu dapat menghasilkan event untuk transaksi yang kemudian gagal. Transactional outbox menghindari dua kondisi tersebut dengan menyimpan hazard dan niat publish pada transaksi yang sama. Relay polling membaca pending berurutan berdasarkan ID, mengirim satu record, menunggu ACK, baru menandai baris published. Retensi outbox membersihkan baris yang sudah published lebih dari 24 jam; pending tidak dibersihkan dengan aturan itu.

Producer meminta `acks=all`. Pada replication factor 1, ini tetap hanya ACK satu broker, bukan replikasi tahan kehilangan mesin. Hasil publish yang masih tidak pasti dipertahankan relay agar versi berikutnya tidak melompati pesan sebelumnya. Bila ACK diterima tetapi proses mati

sebelum penandaan PostgreSQL, event yang sama dapat dipublikasikan kembali. Karena itu semantik aplikasi adalah **at-least-once**, dengan syarat store dan broker tetap tersedia kembali dan data masih dalam retensi; bukan exactly-once end-to-end.

Urutan Kafka hanya dijamin di dalam partisi [6]. Penambahan partisi dapat mengubah pemetaan key, sehingga klaim urutan per hazard tidak diperluas ke perubahan partisi tanpa strategi transisi. Relay tunggal juga merupakan asumsi; multi-relay memerlukan mekanisme claim dan koordinasi tambahan.

Implementasi terkait terdapat pada [services/aggregator/internal/worker/outbox/relay.go](#).

Implementasi terkait terdapat pada [services/aggregator/internal/adapters/outbound/kafka/producer.go](#).

9.3 Efek consumer, idempotensi, dan DLQ

Dashboard-updater dan pemda-portal menyimpan view SQLite dengan primary key hazard ID. Upsert hanya mengganti view apabila version masuk lebih besar daripada version tersimpan. Replay atau versi lama tidak menimpa view terbaru. Notifier memeriksa pasangan hazard dan version dan hanya mengirim alert untuk SIAGA atau AWAS; implementasi pengiriman saat ini berupa log simulasi.

```

1 // Send precedes the durable marker. A crash in between may duplicate delivery.
2 // One sequential consumer owns this SQLite file; it is not a distributed lock.
3 func (s *Service) Handle(ctx context.Context, e contract.Event) error {
4     seen, err := s.Store.Seen(ctx, e.HazardID, e.Version)
5     if err != nil {
6         return err
7     }
8     if seen {
9         return nil
10    }
11    alert := e.Hazard.Severity == "SIAGA" || e.Hazard.Severity == "AWAS"
12    if alert {
13        if err = s.Sender.Send(ctx, e); err != nil {
14            return err
15        }
16    }
17    return s.Store.Record(ctx, e, alert)
18 }

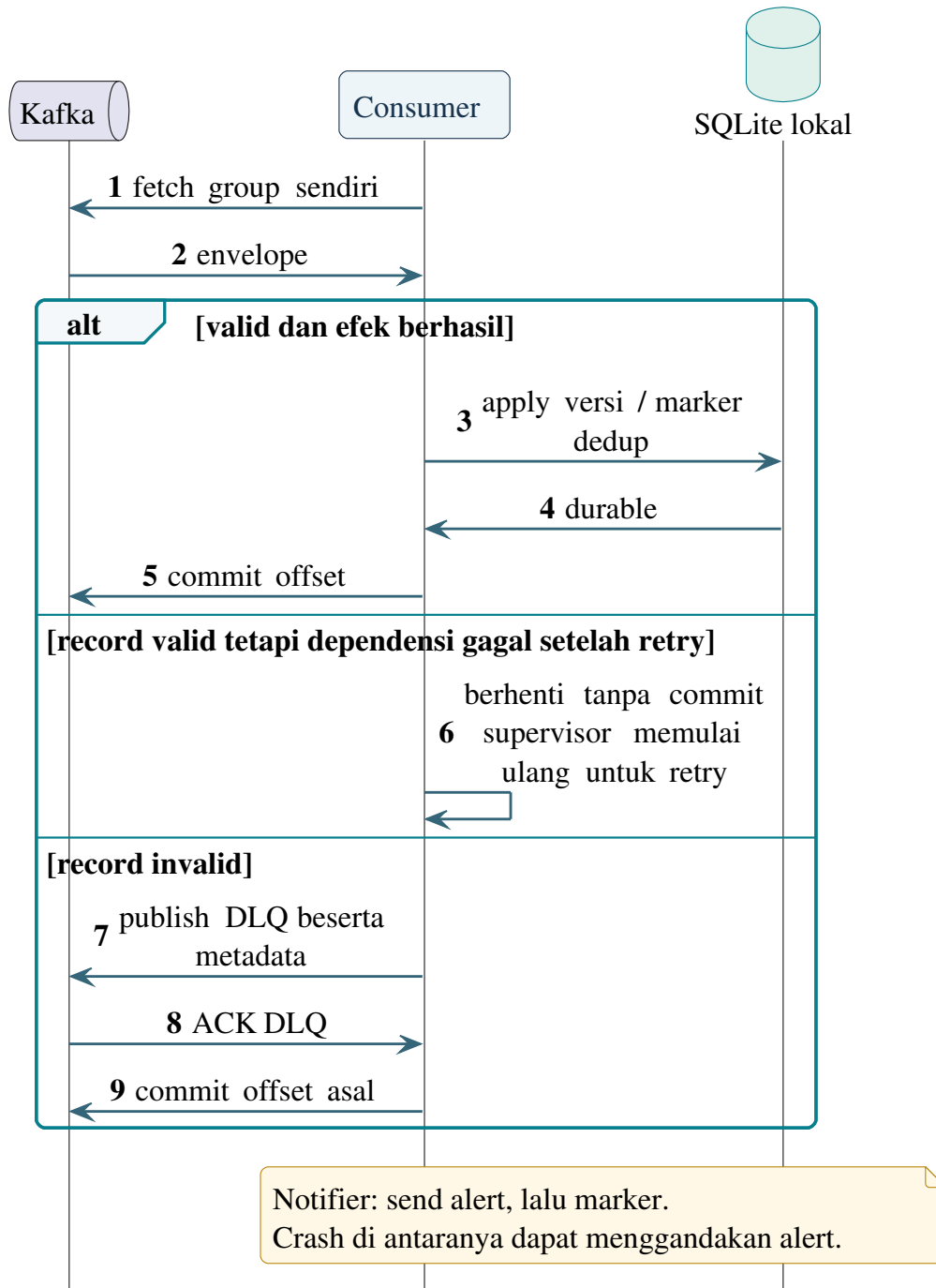
```

Listing 3: Notifier mengirim sebelum mencatat marker persisten; celah crash di antara keduanya diakui sebagai kemungkinan duplikasi.

Consumer menonaktifkan auto-commit. Offset asal di-commit sesudah efek lokal berhasil. Record invalid langsung diarahkan ke DLQ; kegagalan pemrosesan yang dapat dicoba ulang dibatasi oleh jumlah percobaan dan timeout. Setelah percobaan pemrosesan habis, worker berhenti tanpa commit agar record valid dicoba kembali setelah dependensi pulih. Hanya record invalid dipublikasikan ke DLQ dengan identitas group asal, topic, partisi, offset, alasan, jumlah percobaan, dan correlation ID. Offset pesan asal baru di-commit setelah ACK DLQ; kegagalan publish DLQ tidak boleh menghilangkan pesan dengan commit lebih dahulu.

SQLite memakai volume sendiri, WAL, dan synchronous FULL. Efek view persisten serta version guard memberi idempotensi proyeksi. Untuk notifier, crash setelah send tetapi sebelum marker dapat menghasilkan pengiriman ulang. Dedup lokal bukan distributed lock maupun jaminan exactly-once ke perangkat eksternal. Log `record_completed` menunjukkan penanganan record dan commit selesai; record itu bisa berupa duplicate yang diabaikan atau pesan yang masuk DLQ, bukan selalu notifikasi baru.

Implementasi terkait terdapat pada [services/dashboard-updater/internal/store/sqlite.go](#).



Gambar 7: Efek lokal atau ACK DLQ mendahului commit offset. Tidak ada transaksi atomik tunggal yang meliputi PostgreSQL, Kafka, dan efek notifikasi.

Implementasi terkait terdapat pada [services/notifier/internal/application/apply.go](#).

Implementasi terkait terdapat pada [services/notifier/internal/consumer/consumer.go](#).

Pemisahan cabang pada Gambar 7 mencegah gangguan penyimpanan diperlakukan sebagai kerusakan pesan. Record valid tetap diperlukan setelah dependensi pulih, sehingga kegagalan SQLite tidak menjadi alasan memindahkannya ke DLQ dan memajukan offset. Konsekuensinya, group yang terdampak menunggu pemulihan, sementara group lain tetap berjalan. Untuk record invalid, DLQ menyimpan bahan pemeriksaan agar satu pesan yang tidak dapat diproses tidak menahan stream selamanya.

9.4 Alternatif dan trade-off

Pemanggilan HTTP setiap consumer dari Aggregator lebih sederhana di awal, tetapi mengikat pengetahuan producer tentang subscriber dan membuat kegagalan subscriber masuk jalur pengiriman. RabbitMQ dengan fan-out queue per consumer merupakan alternatif wajar untuk distribusi tugas, namun mekanisme replay histori dan lifecycle queue tetap perlu dirancang. Kafka dipilih untuk retensi log dan offset group; biaya memori, operasional broker, duplikasi, dan pengelolaan retention diterima pada demo lokal. Evaluasi ini tidak mengklaim Kafka selalu lebih unggul pada semua beban.

9.5 Bukti penyelesaian

`TestEventPipeline` memeriksa jalur dari mock melalui outbox dan Kafka hingga dashboard, replay event, penolakan versi lama, dedup notifier, dan persistensi setelah restart. Dashboard dihentikan sementara; notifier dan pemda tetap memproses versi berikutnya. Setelah dashboard dimulai lagi, offset yang belum dikonsumsi tersusul. Ketika Kafka dihentikan, proses ingest tetap menghasilkan outbox dengan status pending. Backlog kemudian terkirim setelah broker kembali tersedia.

Uji subscriber baru menggunakan group pemda unik dan SQLite kosong, sehingga bukti replay tidak bergantung pada state run sebelumnya. Aggregator tidak diganti. Konfigurasi group dan volume pemda reguler dipulihkan setelahnya. Catch-up berlaku selama pesan masih tersedia pada retensi; consumer yang offline melewati retensi tidak dijamin mendapatkan seluruh histori. Replikasi satu broker juga tidak melindungi dari kehilangan seluruh volume Kafka.

Bukti eksekusi tersedia pada [docs/evidence/integration/events-fresh-group.txt](#).

Bukti eksekusi tersedia pada [docs/evidence/integration/event-trace.jsonl](#).

10 Ringkasan Verifikasi dan Batas Hasil

Verifikasi memeriksa kesesuaian perilaku sistem terhadap persyaratan melalui uji unit, analisis statis dengan go vet, integrasi PostgreSQL, regresi stack, dan load test. Tabel berikut merangkum skenario serta hasilnya. Rincian pemetaan tiap kriteria tersedia pada Lampiran A, sedangkan lokasi artefak terdapat pada Lampiran B.

10.1 Skenario dan hasil akhir

Kriteria	Skenario yang diperiksa	Hasil
P1	Pemetaan sumber, korelasi tsunami, serta penambahan field saat runtime.	Field tambahan dipertahankan; record lama dan baru dapat dibaca tanpa restart service atau perubahan skema tabel.
P2	Pembacaan seismic dan volcanic bersamaan, beban sustained, serta outage dan recovery PVMBG.	P95 seismic 12,51 ms; sedikitnya 50 TCP selama 87,78 s; error bisnis di luar 429 sebesar 0%. Rincian pada Bagian 6.
P3	Kredensial lintas domain, scope, proyeksi field, expiry alami 60 s, dan refresh CLI.	Kredensial silang ditolak; Media tidak memperoleh data mentah; token lama kedaluwarsa ditolak dan token baru diterima.
P4	Rebuild service secara independen, akses melalui API pemilik storage, dan persistensi setelah restart.	Regresi lulus untuk skenario isolasi dan pemulihan yang diuji. Batas kepemilikan diperiksa melalui Compose serta adapter service.
P5	Distribusi ke consumer independen, replay, dedup, subscriber baru, serta pemulihan broker dan consumer.	Stream diterima sesuai group; view tidak ditimpa versi lama; event pending diproses setelah dependensi pulih.
U7	HTTP pertama gagal lalu retry berhasil, beserta pembatalan, timeout, dan sanitasi log.	Setiap percobaan memiliki log latency sendiri dengan correlation ID yang sama; seluruh tes Client API dan go vet lulus.

Regresi stack lulus dalam 306,263 s. Uji module dan go vet serta integrasi PostgreSQL juga lulus, dengan transkrip pada [docs/evidence/reliability-2026-10-08/modules.txt](#), [docs/evidence/final-2026-10-08/postgres.txt](#), dan [docs/evidence/final-2026-10-08/regression.txt](#). Pengujian memakai Docker Desktop lokal; konfigurasi beban, lingkungan pengukuran, serta definisi metrik dijelaskan pada Bagian 6. Penolakan HTTP 429 dilaporkan terpisah dari respons sukses dan tidak dihitung sebagai throughput sukses.

```

--- PASS: TestHealth (0.04s)
--- PASS: TestMockContracts (6.65s)
--- PASS: TestPVMBGSchemaAndOutage (10.23s)
--- PASS: TestTokenRotationAndAuthorization (0.19s)
--- PASS: TestJWTClaimValidation (0.02s)
--- PASS: TestPersistenceAndDependencyRecovery (27.09s)
--- PASS: TestStructuredLogsDoNotExposeCredentials (0.49s)
--- PASS: TestIngestPipeline (50.05s)
--- PASS: TestEventPipeline (81.85s)
--- PASS: TestQueryIntegration (0.66s)
--- PASS: TestDynamicSchemaAndStaleAPI (26.34s)
--- PASS: TestNaturalExpiryAndFieldCLI (68.77s)
--- PASS: TestIndependentRebuild (26.24s)
--- PASS: TestLargeEventDeliveryAndPermanentRejection (6.48s)
PASS
ok      command-line-arguments  306.263s

```

Listing 4: Petikan hasil regresi stack. Transkrip lengkap tersedia pada artefak pengujian.

10.2 Ketahanan ukuran event dan pemulihan

Pengujian PostgreSQL menerima envelope 4 MiB dikurangi satu byte serta tepat 4 MiB, mengarahkan envelope yang melebihi batas satu byte, dan tetap memproses record berikutnya. Backlog 600 record diuji dengan kegagalan di tengah respons. Checkpoint tidak maju saat gagal dan pengulangan tidak menghasilkan outbox ganda.

Pengujian lintas service mengalirkan event 4 MiB utuh melalui relay, Kafka, ketiga consumer, dan API menggunakan fixture administratif. Pengujian penyimpanan SQLite memeriksa bahwa kegagalan storage tidak memindahkan event valid ke DLQ atau memajukan offset sebelum pemulihan. Hasil ini mendukung mekanisme retry dan pemulihan pada skenario yang diuji, dengan batas retensi serta semantik pengiriman pada Bagian 9.

10.3 Verifikasi instrumentasi HTTP

Client API mencatat satu log `upstream_request` untuk setiap pemanggilan HTTP.Do oleh adapter. Timer dimulai ulang pada setiap percobaan dan mencakup pembacaan, decode, serta penutupan body respons. Log memuat correlation ID, nomor percobaan, latency, status HTTP, dan kategori hasil. Status 0 menunjukkan respons HTTP belum diperoleh. URL, kredensial, isi respons, dan pesan galat upstream tidak dicatat.

`TestRetryLogsEachAttempt` menggagalkan percobaan pertama lalu memberikan respons valid pada percobaan kedua. Tes memeriksa dua log dengan nomor 1 dan 2, correlation ID yang sama, hasil transport error diikuti sukses, serta deadline yang tidak diperpanjang. Tes lain memeriksa batas retry, pembatalan, timeout, respons invalid, dan penutupan body. Seluruh tes module Client API dan go vet lulus. Temuan U7 tentang latency retry yang tergabung telah ditutup melalui pengujian ini.

Implementasi terkait terdapat pada [services/client-api/internal/adapter/outbound/aggregator/attempt_test.go](#).

Implementasi terkait terdapat pada [docs/evidence/http-attempts-2026-10-08/README.md](#).

10.4 Bootstrap pada volume kosong

Pengujian mengekstrak source commit ke direktori terpisah tanpa konfigurasi lokal lama. Bootstrap membuat 12 berkas konfigurasi dan kunci baru; pemanggilan kedua mempertahankannya tanpa rotasi. Project Compose memakai network tersendiri serta enam volume yang belum ada sebelum pengujian.

Migrasi mencapai versi 3 dengan status dirty false. API menyajikan 42 hazard dengan kedua sumber HEALTHY, dan permintaan raw oleh Media ditolak dengan 403. Dashboard serta Pemda masing-masing menerima 42 snapshot. Notifier mencatat 46 pemrosesan karena hazard dapat mempunyai beberapa versi. Ketiganya mencakup seluruh ID hazard yang diperiksa pada Canonical Store. Rincian hasil tersedia pada [docs/evidence/final-2026-10-08/bootstrap/result.json](#).

Setelah pengujian, volume project uji dibersihkan dan stack utama dijalankan kembali. Pemeriksaan memastikan volume, isi kredensial, dan satu hazard acuan stack utama tetap tersedia. Uji ini membuktikan startup dengan data persisten kosong dan konfigurasi baru pada host yang sudah memiliki Docker serta cache build; bukan instalasi host dari nol.

10.5 Cakupan bukti dan pemeriksaan akhir

Regresi stack dan load test dijalankan pada implementasi `a465139`, yang sudah mencakup instrumentasi latency per percobaan HTTP. Tes module Client API memeriksa cabang retry secara terarah, sedangkan regresi dan k6 memeriksa perilaku stack setelah dibangun ulang. Masing-masing artefak tetap dicatat menurut cakupan pengujiannya.

Regresi dan load test memakai volume stack utama, sedangkan bootstrap diverifikasi terpisah pada enam volume kosong. Selain skenario outage 20 detik, pengujian tambahan telah menjalankan outage 600 detik dengan recovery 90 detik dan memeriksa pemulihan tanpa restart. Pengujian satu host juga belum membuktikan failover lintas mesin atau kapasitas deployment lain.

Pemeriksaan secret final menggunakan Gitleaks 8.24.2 pada revisi `48987b7` memindai 58 commit dan tidak menemukan kebocoran. Bukti tersedia pada `docs/evidence/reliability-2026-10-09/secret-scan.txt`. Status kelengkapan persyaratan tersedia pada [docs/requirements-audit.md](#).

Verifikasi CI hosted: workflow `foundation` run `#21` berhasil pada branch `main` untuk commit `013099b`. [Tautan langsung run GitHub Actions #21](#).

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Muhammad Alfansya/13523005
 Dzaky Aurelia Fawwaz/13523065
 Fakultas : Sekolah Teknik Elektro dan Informatika
 Kode Mata Kuliah : IF4031
 Nama Mata Kuliah : Arsitektur Aplikasi Terdistribusi
 Judul Tugas : Tugas Besar AAT Milestone 1 arsitektur aplikasi terdistribusi

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Adapun bentuk bantuan AI yang digunakan dalam proyek adalah sebagai berikut:

Kegiatan	Bentuk bantuan AI dalam proyek
Diskusi arsitektur	Menelaah rancangan BNPB, menjelaskan trade-off, memeriksa pembagian komponen, dan mengidentifikasi kemungkinan overengineering atau ketidaksesuaian kontrak.
Struktur repository	Membantu membuat folder, README komponen, dan dokumentasi kontrak sebagai acuan implementasi.
Implementasi	Membantu menghasilkan dan menyunting kode berdasarkan rancangan dan arahan kelompok, termasuk bagian ingest Aggregator, autentikasi, outbox, consumer, dan infrastruktur lokal.
Review dan integrasi	Meninjau implementasi query, Client API, CLI Tim Lapangan, dan k6, lalu mengusulkan atau menerapkan perbaikan agar bagian-bagian tersebut dapat diintegrasikan.
Pengujian dan diagnosis	Membantu menyusun serta menjalankan tes, membaca log, menganalisis hasil k6, dan memperbaiki masalah yang ditemukan.
Dokumentasi	Membantu menyusun draf laporan, sumber LaTeX, diagram berbasis teks, penjelasan komponen, dan ringkasan bukti pengujian. Kelompok memberikan koreksi isi, struktur, dan gaya penulisan.

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 8 Oktober 2026



Muhammad Alfansya
13523005

Bandung, 8 Oktober 2026



Dzaky Aurelia Fawwaz
13523065

A Matriks Kriteria Spesifikasi

ID di bawah mengikuti urutan kriteria pada spesifikasi M1. “Lokal” berarti telah diperiksa oleh suite yang dirujuk, bukan pengganti pelaksanaan demo sinkron. Rujukan U1 hingga U7 mengikuti ketentuan implementasi umum. Indeks bukti E1 hingga E12 dijelaskan pada Lampiran B.

ID	Kriteria dan mekanisme	Verifikasi dan bagian laporan
U1	Panggilan jaringan; tidak berbagi business logic lintas service.	Module terpisah, Compose, dan kode adapter; Bagian 2.
U2	Satu pemilik storage; client-api melalui API Aggregator.	Konfigurasi network pada Compose beserta adapter HTTP; Bagian 8.
U3	Dockerfile per service dan satu orkestrasi.	Dockerfile delapan service, Compose, serta bootstrap enam volume kosong; E12 dan petunjuk start.
U4	Kegagalan dan delay benar-benar dipicu.	Outage PVMBG dan Kafka, restart dependensi, rebuild notifier; E2, E3, dan E4.
U5	Kredensial sumber tidak berlaku lintas instansi.	TestMockContracts ; E2.
U6	Secret dari konfigurasi lokal dan contoh environment.	Generator secret, ignore, <code>.env.example</code> ; Bagian 7.
U7	Health, log terstruktur, correlation ID, dan latency tiap panggilan keluar.	Tersedia. Tes Client API memeriksa log per percobaan HTTP, correlation ID, deadline bersama, dan sanitasi; hasil pada Bagian 10.
P1.1	Pemetaan kanonik kedua sumber.	Unit mapper dan TestIngestPipeline ; E1 dan E2.
P1.2	Schema v2 dipicu saat runtime.	Endpoint admin dalam TestDynamicSchemaAndStaleAPI ; E2.
P1.3	Ingest tetap valid tanpa perubahan kode atau restart.	Record baru terbaca; ID container dan versi DDL tetap; E2.
P1.4	Kebijakan unknown field dinyatakan dan konsisten.	Extra ke attributes, validasi dan karantina; Bagian 5, E1 dan E2.
P1.5	Mekanisme serta perubahan yang didukung dan yang tidak didukung.	Tolerant reader additive-only dan batas tipe; Bagian 5.
P2.1	Seismic p95 < 300 ms saat volcanic paralel dan delay 3 s.	12,51 ms; E9.
P2.2	Minimal 50 koneksi, minimal 60 s, error < 1% di luar 429.	50 TCP selama 87,78 s; error 0%; 33.718 bisnis 429 dilaporkan; E9.
P2.3	Seismic tetap dilayani, volcanic stale atau unavailable, lalu recovery.	Outage 600 s; seismic 14.281/14.281 sehat; volcanic basi setelah deteksi dan pulih tanpa restart; E12.
P2.4	Model konkurensi, pencegahan tiga kegagalan, konfigurasi load.	Bagian 6; script k6 dan runner.
P3.1	BMKG dan PVMBG menolak kredensial silang.	HTTP 401 atau 403 pada suite fondasi; E2.
P3.2	Media hanya ringkasan dan raw eksplisit ditolak.	TestQueryIntegration ; E2.

ID	Kriteria dan mekanisme	Verifikasi dan bagian laporan
P3.3	Expiry alami, refresh otomatis, token lama ditolak.	TTL 60 s, sesi CLI 65 s, pemeriksaan 401 dan 200; E2.
P3.4	Tidak ada secret yang ditulis langsung dalam kode atau tersimpan dalam commit.	Bootstrap, contoh env, ignore, test log; scan final 58 commit hingga 48987b7 tanpa temuan; docs/evidence/reliability-2026-10-09/secret-scan.txt.
P3.5	Alasan keamanan mekanisme dan batasnya.	Domain trust, claim, allowlist, replay, transport, dan revocation; Bagian 7.
P4.1	Penghentian, build ulang, dan pengaktifan kembali satu komponen tanpa deploy yang lain.	Notifier rebuild, baca API tetap berhasil, timestamp start lain tetap; E2.
P4.2	Storage fleksibel tanpa DDL untuk field baru; record lama dan baru tersedia bersama.	Query record sebelum dan sesudah confidence; E2.
P4.3	Canonical Store hanya diakses aplikasi pemilik.	Network, kepemilikan secret, dan query melalui HTTP; Bagian 8.
P4.4	Alasan batas service dan minimal dua alternatif storage.	Perbandingan PostgreSQL dengan JSONB, document store, dan kolom tetap; Bagian 8.
P5.1	Hazard baru dikirim ke broker.	Commit outbox, publish ACK, dan konsumsi snapshot; E3.
P5.2	Minimal dua consumer independen.	Dashboard dan notifier dengan group terpisah; E3.
P5.3	Satu consumer mati tidak menahan yang lain; jelaskan catch-up.	Dashboard dihentikan dan diaktifkan kembali, versi susulan, retensi; E3.
P5.4	Subscriber baru tanpa perubahan producer.	Group pemda unik dan store kosong; Agregator tidak diganti; E3.
P5.5	Broker, pola, semantik pengiriman, dan idempotensi.	At-least-once, version guard, marker, batas notifier; Bagian 9.

B Indeks Bukti dan Jejak Eksekusi

Seluruh berkas E1 hingga E8 berada di `docs/evidence/integration/` pada revisi acuan. Artefak disimpan sebagai teks dan JSON agar dapat ditinjau tanpa bergantung pada screenshot. Potongan yang dicetak merupakan petikan atau rekam berlabel; output lengkap tetap menjadi acuan.

ID	Berkas	Isi
E1	<code>docs/evidence/integration/modules.txt</code>	Uji unit dan go vet sembilan module dan bootstrap.
E2	<code>docs/evidence/integration/regression.txt</code>	Regresi lengkap termasuk auth, ingest, query, drift, dan rebuild.
E3	<code>docs/evidence/integration/events-fresh-group.txt</code>	Replay subscriber baru, DLQ, dedup, dan outage broker.
E4	<code>docs/evidence/integration/load/seismic-only.json</code> , <code>docs/evidence/integration/load/sustained.json</code> , <code>docs/evidence/integration/load/outage.json</code>	Summary k6 awal. Angka akhir laporan memakai E9.
E5	<code>docs/evidence/integration/load/connections.json</code>	Sampel waktu host dan koneksi TCP teramati.
E6	<code>docs/evidence/integration/events-trace.jsonl</code>	Trace nyata dari mock sampai consumer.
E7	<code>docs/evidence/integration/postgres.txt</code>	Integrasi transaksi dan query PostgreSQL.
E8	<code>docs/evidence/integration/services.txt</code>	Snapshot status container akhir sesi pengujian.

B.1 Bukti pengujian akhir

E9 merujuk `docs/evidence/final-2026-10-08/README.md` beserta JSON k6, sampel koneksi, dan konfigurasi pada subfolder load. Angka P2 memakai pengujian final ini.

E10 merujuk `docs/evidence/final-2026-10-08/regression.txt` dan `docs/evidence/final-2026-10-08/postgres.txt`. Regresi stack lulus dalam 306,263 s. Bukti unit seluruh module dan scan histori yang cakupannya lebih awal tetap tersedia pada `docs/evidence/reliability-2026-10-08/modules.txt` serta `docs/evidence/reliability-2026-10-08/secret-scan.txt`.

E11 merujuk `docs/evidence/http-attempts-2026-10-08/README.md`, yang memuat hasil tes module Client API dan go vet untuk verifikasi latency per percobaan HTTP. Bukti ini tidak menggantikan pengukuran k6 pada E9.

E12 merujuk `docs/evidence/final-2026-10-08/long-outage/result.json`, `docs/evidence/final-2026-10-08/long-outage/summary.json`, dan `docs/evidence/final-2026-10-08/bootstrap/result.json`. Artefak ini memeriksa outage 600 detik, pemulihan tanpa restart, serta bootstrap dengan kredensial dan volume baru. Manifest pada `docs/evidence/final-2026-10-08/manifest.json` mencatat hash artefak serta runner.

B.2 Contoh trace event nyata

Correlation ID: 2a877ebd-3298-4ba1-8f24-8bed0d80d6e3.

Service	Pesan log	Latency (ms)
bmkg-mock	http_request	100.426
aggregator	poll_complete	118
aggregator	outbox_published	1
pemda-portal	record_completed	6
notifier	notification_simulated	–
dashboard-updater	record_completed	5
notifier	record_completed	4

Correlation ID dibentuk pada polling lalu diteruskan ke mock, transaksi dan envelope, publish, dan consumer. Query client memiliki correlation ID tersendiri; query tidak dianggap sebagai penyebab ingest. Pada log consumer, latency menunjukkan waktu pemrosesan record sampai commit, bukan latency ujung-ke-ujung sejak kejadian bencana.

B.3 Asal potongan kode dan gambar

Potongan kode di `assets/code/` diekstrak dari revisi acuan, dengan lokasi dan hash pada `assets/provenance.json`. Sumber enam diagram berada di `diagrams/`; grafik performa dibangkitkan dari JSON bukti final. Semua dapat dibuat ulang dengan `scripts/refresh-assets.py`. Gambar bukan hasil tangkapan layar implementasi atau bukti tambahan yang dibuat-buat.

Pustaka

- [1] Tim pengampu IF4031. *Tugas Besar – Milestone 1*, berkas yang diberikan kelompok: IF4031 – Tugas Besar – M1 (1).pdf, 29 halaman, 2026. Bagian spesifikasi, problem set, dan laporan, hlm. 4–29.
- [2] Tim pengampu IF4031. *Tugas Besar – Dokumen Terpusat*, berkas yang diberikan kelompok: IF4031 – Tugas Besar – Dokumen Terpusat (1).pdf, 10 halaman, 2026.
- [3] Kelompok AkuAnakTehat. *Rancangan Arsitektur BNPB*, dokumen kerja awal, September 2026. Digunakan sebagai bahan alasan desain; klaim implementasi diverifikasi kembali.
- [4] Kelompok AkuAnakTehat. *Repository implementasi dan bukti M1*, revisi f6e99cc. <https://github.com/WwzFwz/AkuAnakTehat/tree/f6e99cc10fec64042eb184c9d7f92d14cb22fd23>.
- [5] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation: JSON Types*. <https://www.postgresql.org/docs/16/datatype-json.html>. Diakses 8 Oktober 2026.
- [6] Apache Kafka. *Kafka 3.9 Design: Message Delivery Semantics*. <https://kafka.apache.org/39/design/design/>. Diakses 8 Oktober 2026.
- [7] Grafana Labs. *k6 Built-in Metrics*. <https://grafana.com/docs/k6/latest/using-k6/metrics/reference/>. Definisi metrik dibaca bersama script k6 versi 0.57.0 yang digunakan dalam pengujian.